교육부 Ministry of Education

NRF 한국연구재단 National Research Foundation of Korea

공유교과목 [딥러닝]

Neural Network Model

COSS Convergence and Open Sharing System 인공지능

전남대학교 CHONNAM NATIONAL UNIVERSITY

성균관대학교 SUNGKYUNKWAN UNIVERSITY

KNU 경북대학교 KYUNGPOOK NATIONAL UNIVERSITY

전주대학교 JEONJU UNIVERSITY

서울시립대학교 UNIVERSITY OF SEOUL

서울과학기술대학교 SEOUL NATIONAL UNIVERSITY OF SCIENCE & TECHNOLOGY

영진전문대학교 YEUNGJIN UNIVERSITY

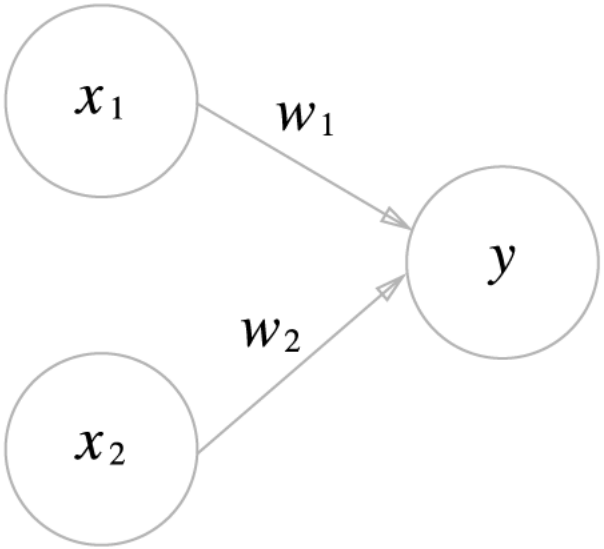
Neural Network

Neural Network

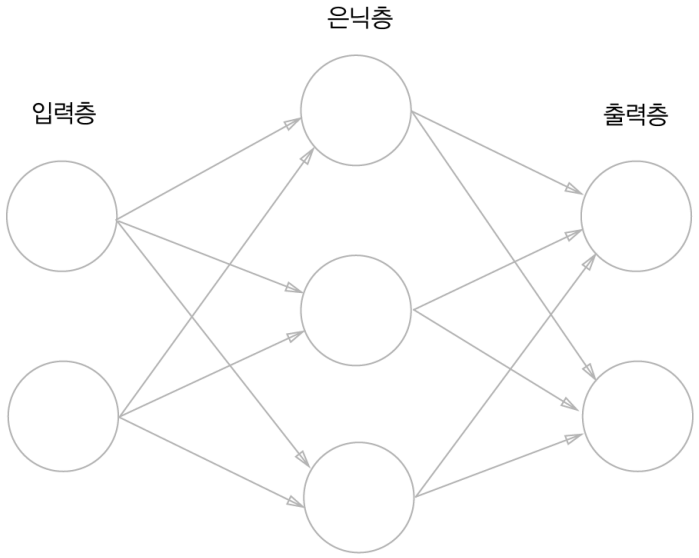
From "Perceptron" to "Neural Network"

- A (single layer) perceptron refers to a single layer neural network that works as a binary classifier

- Neural network refers to an entire class of models which learn using **advanced algorithms** (e.g., hidden layers; activation function; error back propagation)



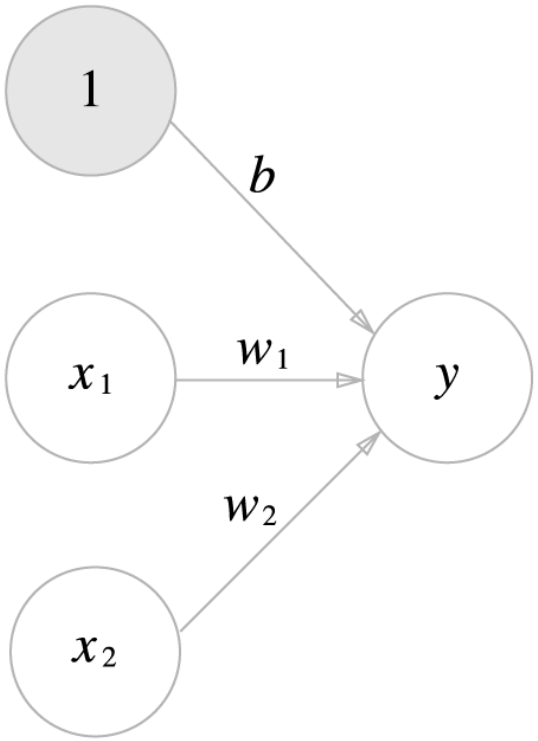
Perceptron



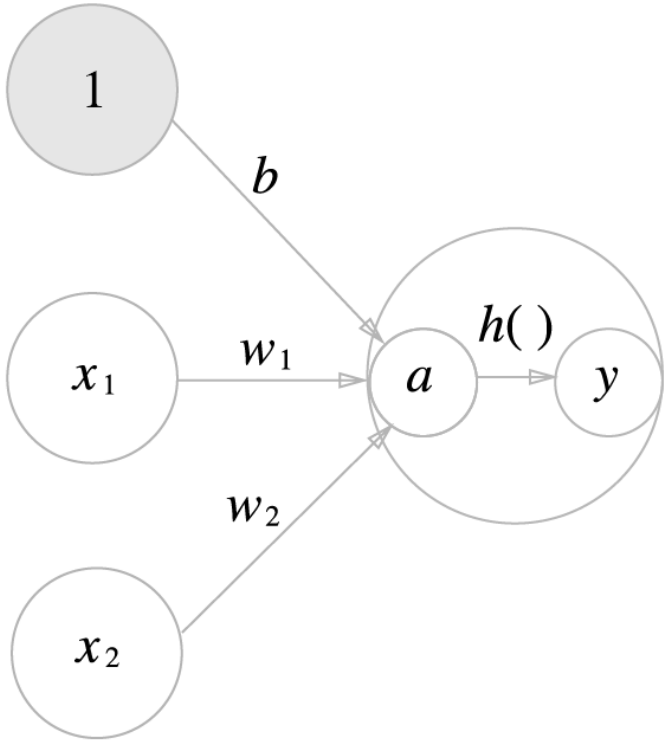
Neural Network

Activation function

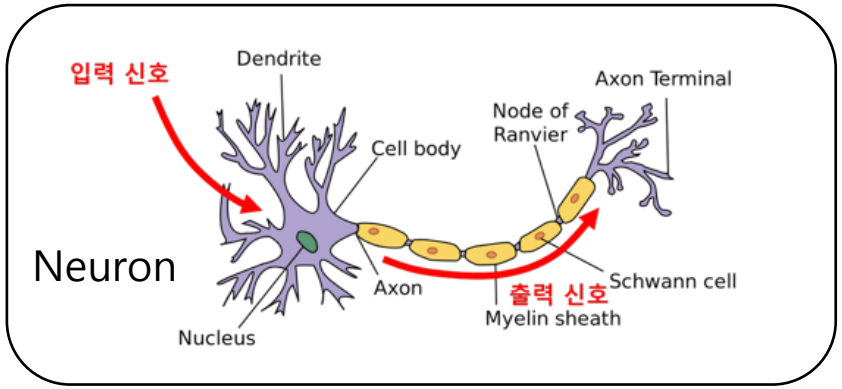
- A function that converts the sum of the input signal into an output signal
- Plays a role in determining whether the sum of input signals triggers activation



Perceptron



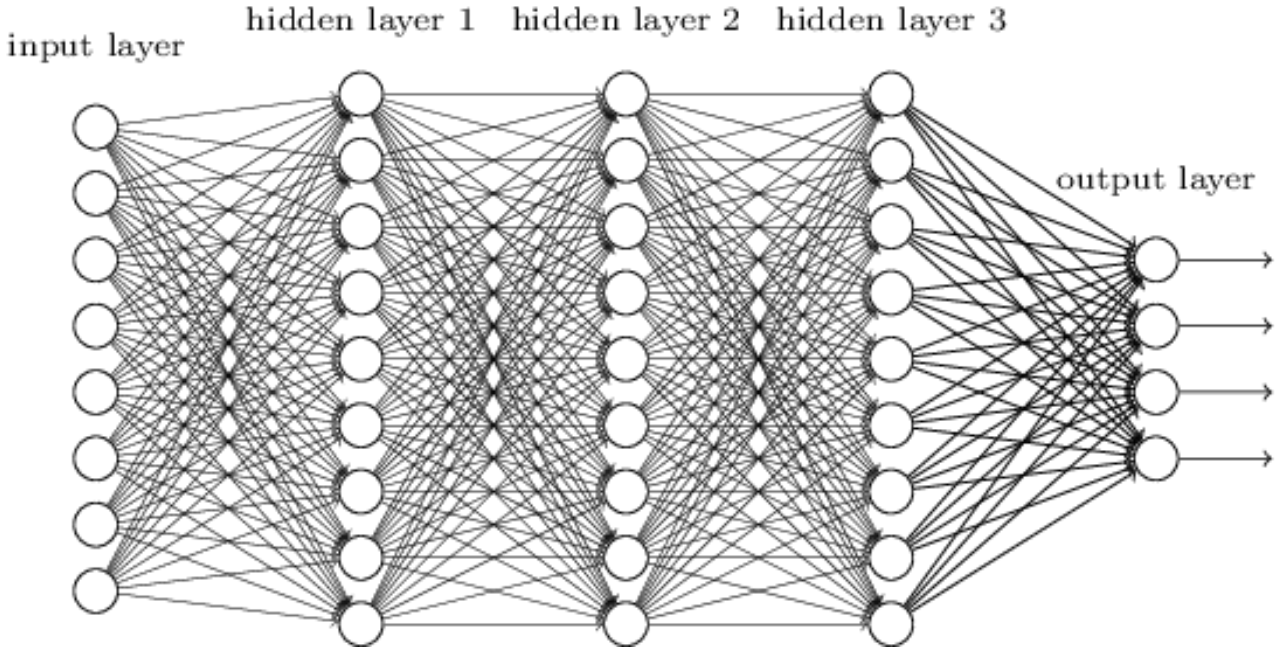
Activation function *h()*



Neural Network

Properties of Activation functions

- Non-linearity



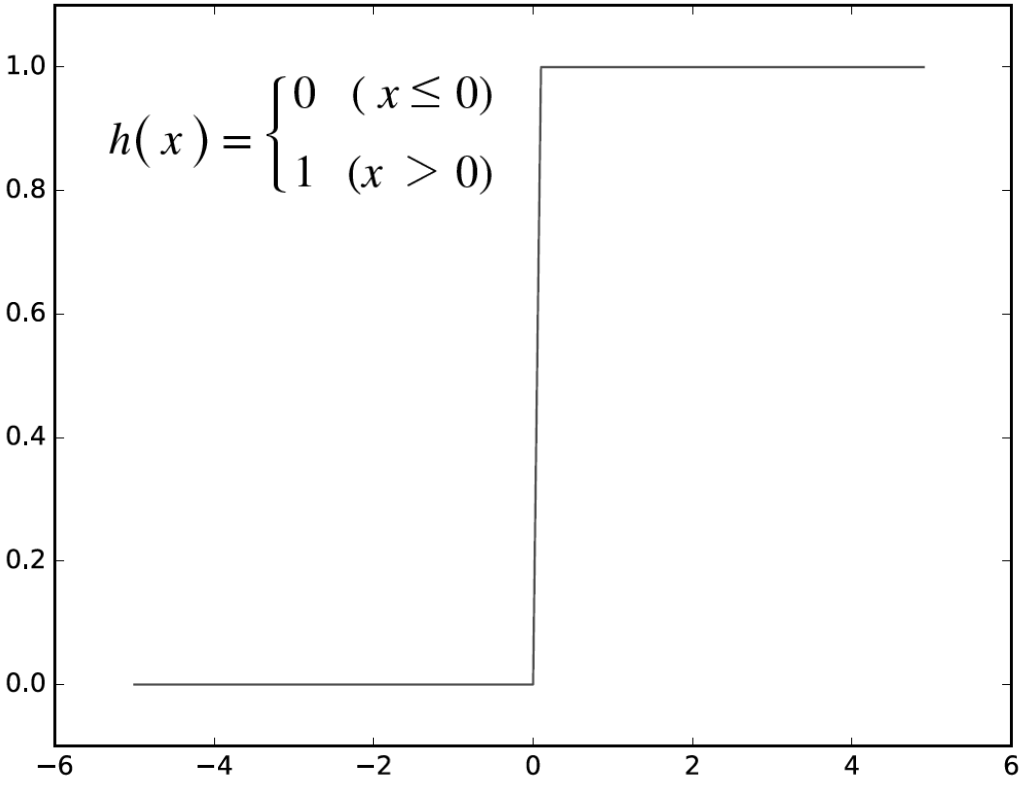
- If we use linear activation function
  $\boldsymbol{h}(x) = cx$, then $\boldsymbol{y}(x) = \boldsymbol{h}(\boldsymbol{h}(\boldsymbol{h}(x))) = c^3x$
  활성화 함수

- Using a linear activation function
  doesn't give you the benefit of having multiple layers

Neural Network

Activation function (1) Step Function

- Simplest example of an activation function
- Determines whether to activate the weighted sum based on the threshold
- When used in an artificial neural network, it causes problems in the weight update process where the gradient is infinite (x=0)
- Therefore, it suits only for a single layer perceptron, and it not suitable for deep learning



Neural Network

Activation function (2) Sigmoid Function

- A function with the shape of a sigmoidal curve

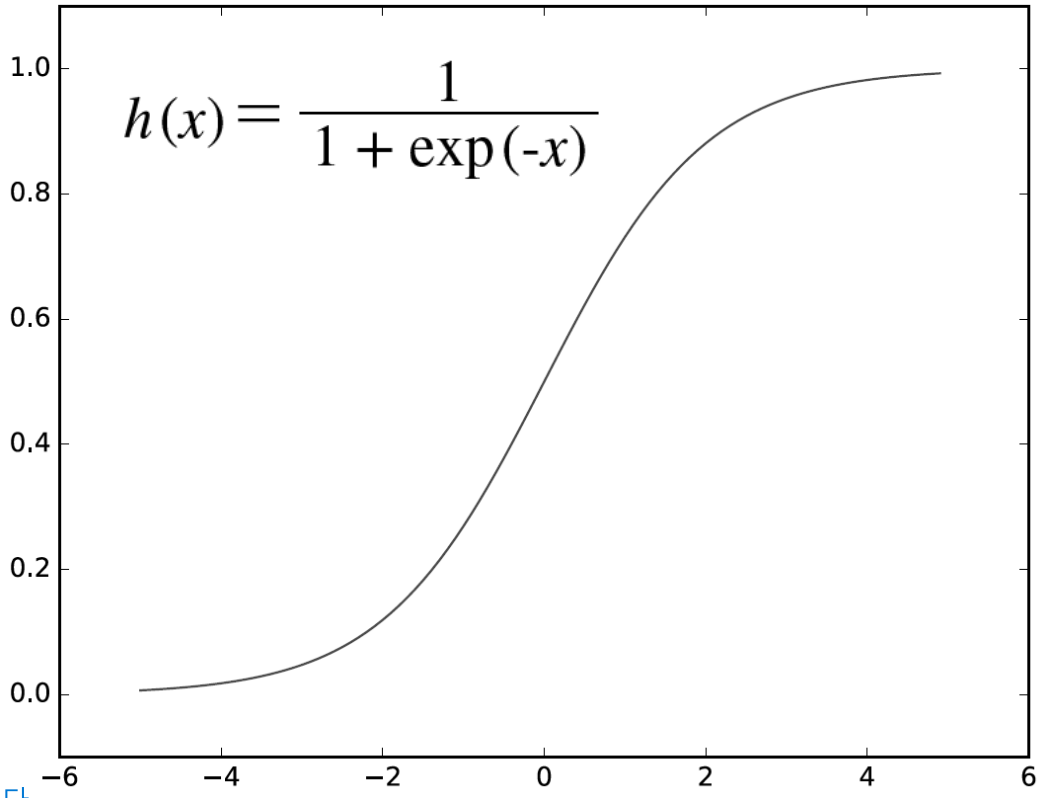
- Allows non-linear interpretation
  (which is important for neural network)

- The domain is the whole real number,
  but returns a finite value between the finite interval

- Vanishing gradient problem occurs
  when the absolute value of the domain increases

기울기 소실 문제

시그모이드 함수에서 층을 미분한 것이 0에 한없이 가까워지기에 기울기 소실 문제가 발생할 수 있다

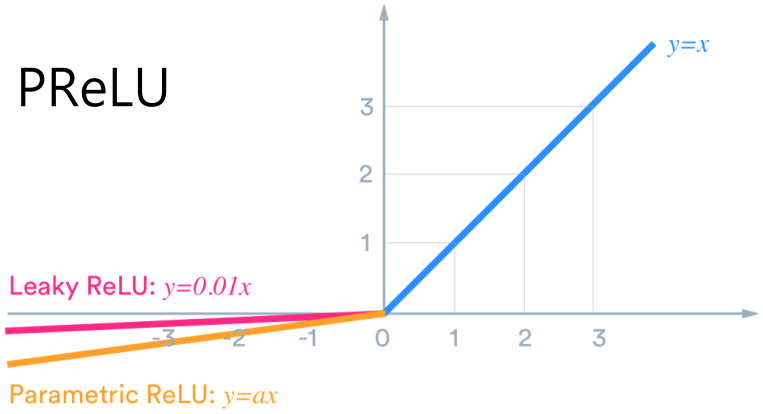


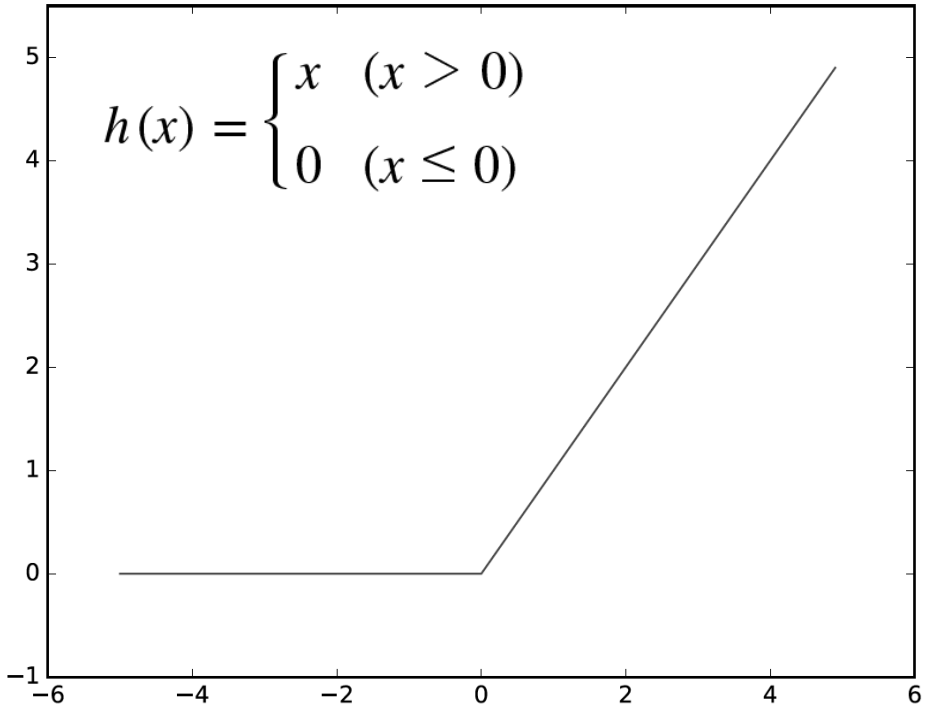
Activation function (3) ReLU Function

- Outputs 0 if the input value is less than 0, and outputs the input value as it is (linear function) if it is greater than 0

- Solves the gradient vanishing problem, which is a disadvantage of sigmoid function

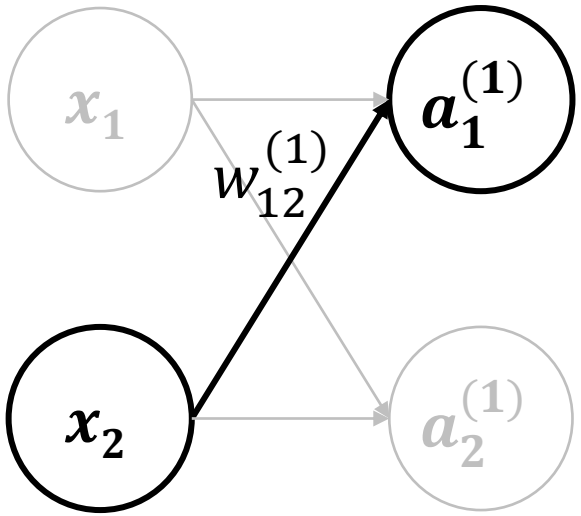
- A dying ReLU (dead neuron) problem occurs in which nodes with a negative weighted sum are not activated again
  → solution: leakyReLU, PReLU





Neural Network

Notations



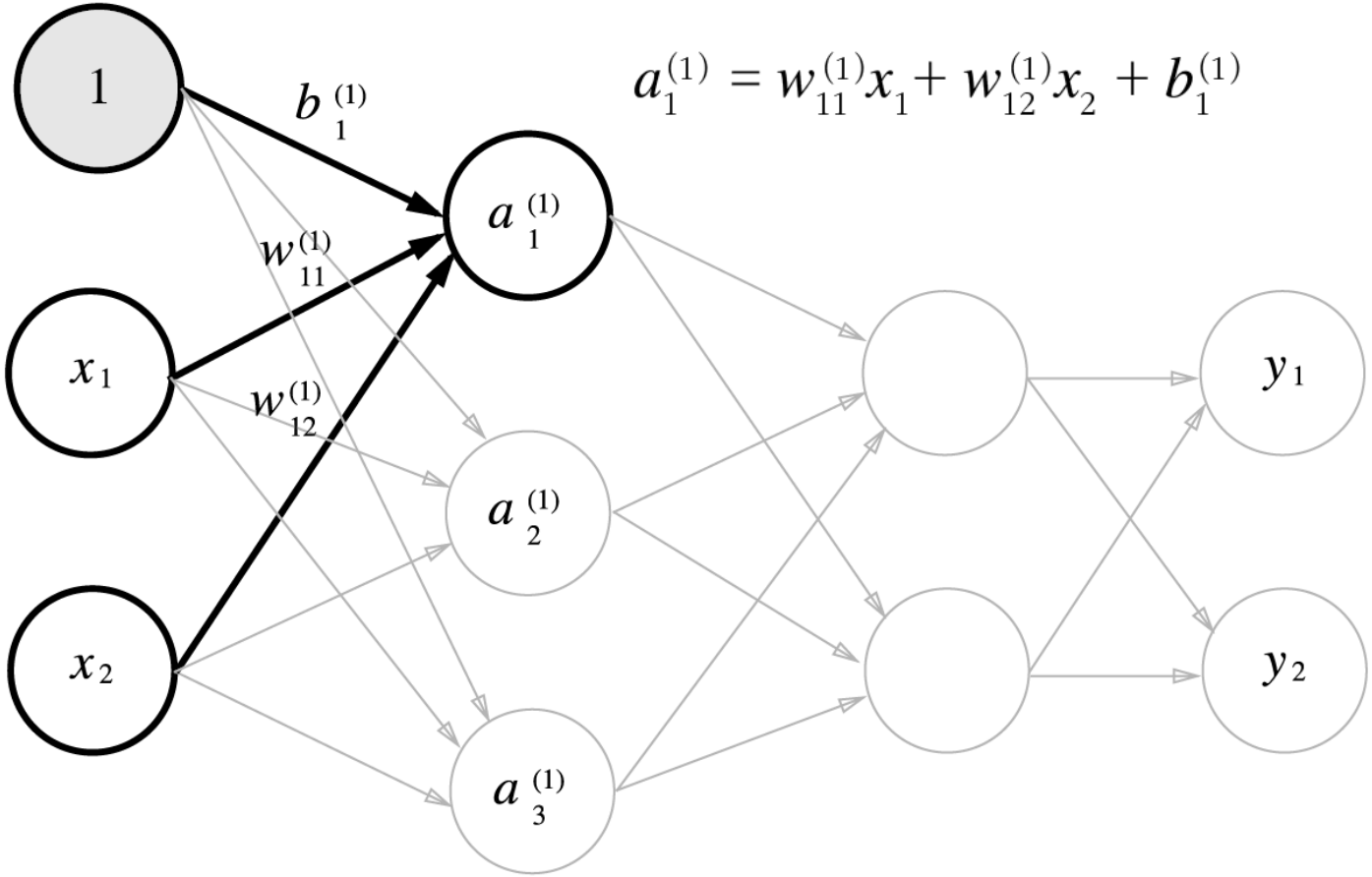
$$w_{12}^{(1)}$$

1st layer's weight

1st neuron in the next layer

2nd neuron in the front layer

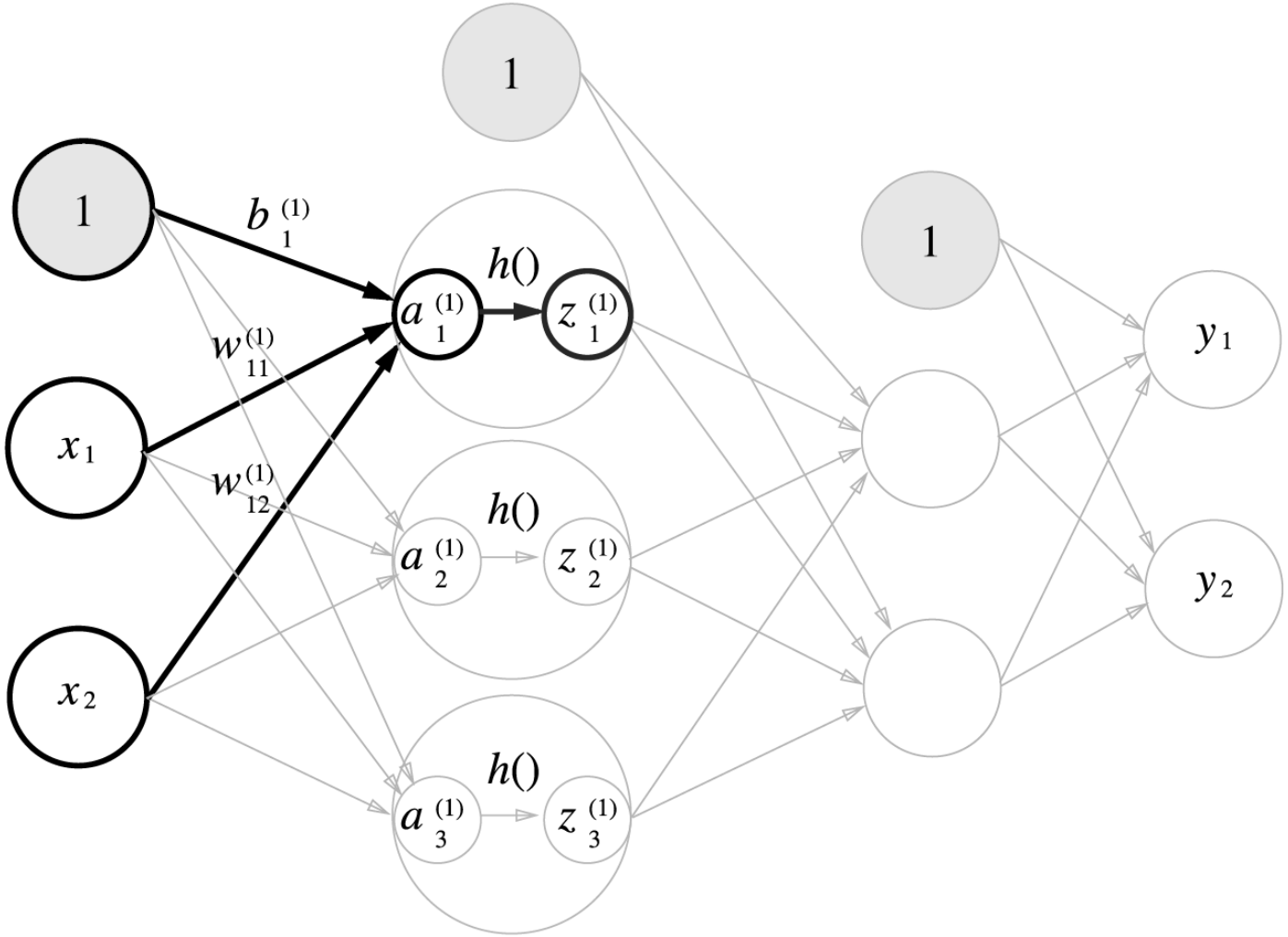
Feedforward



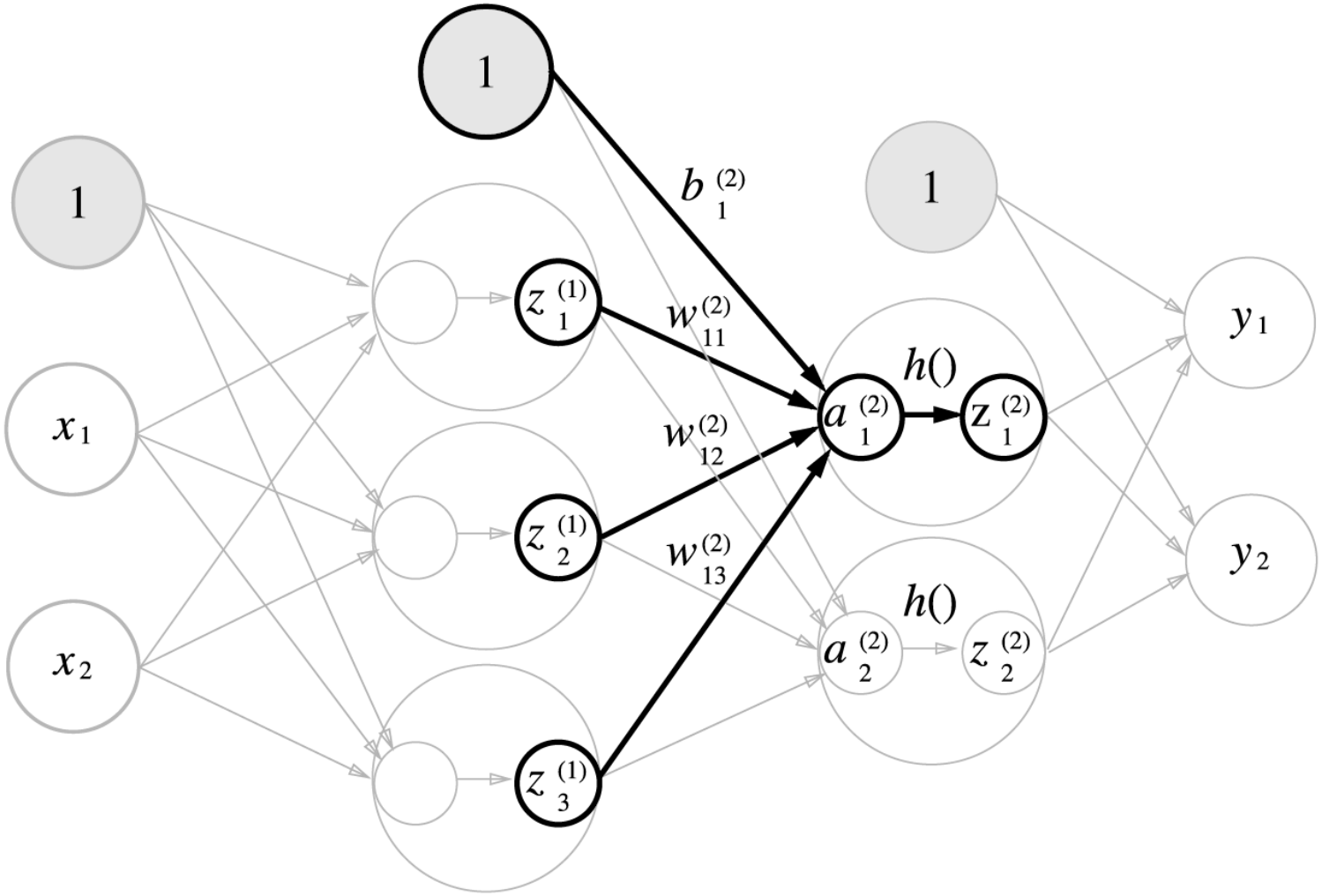
$$a_1^{(1)} = w_{11}^{(1)}x_1 + w_{12}^{(1)}x_2 + b_1^{(1)}$$

Neural Network

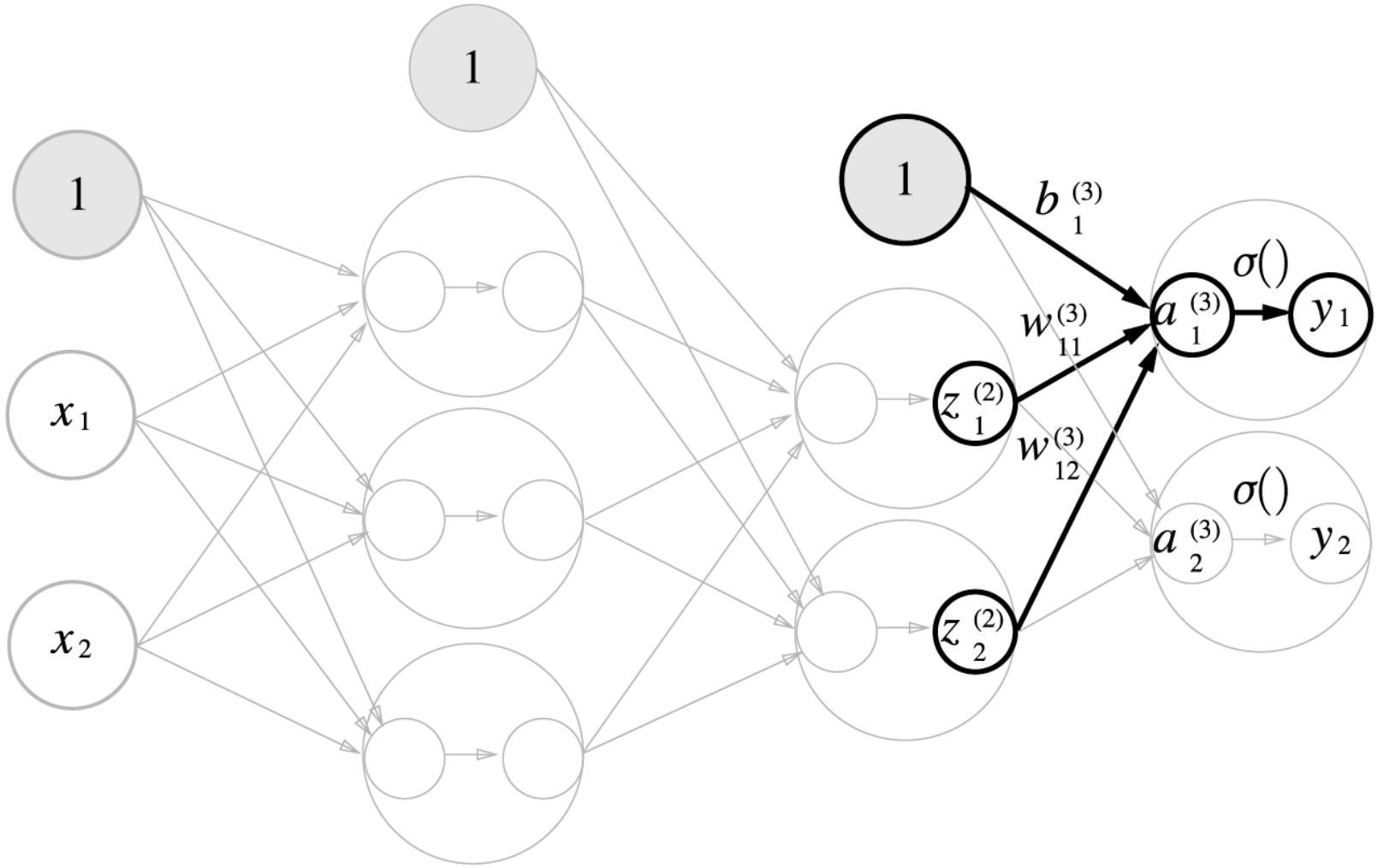
Feedforward



Feedforward



Feedforward

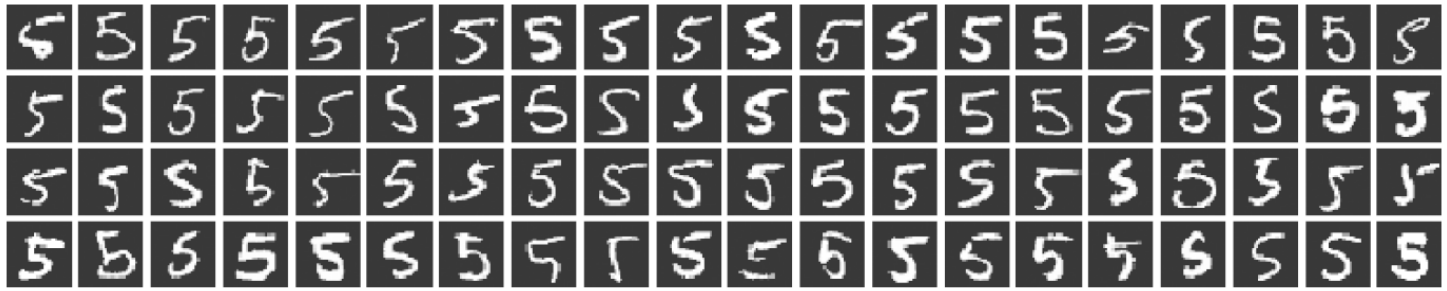


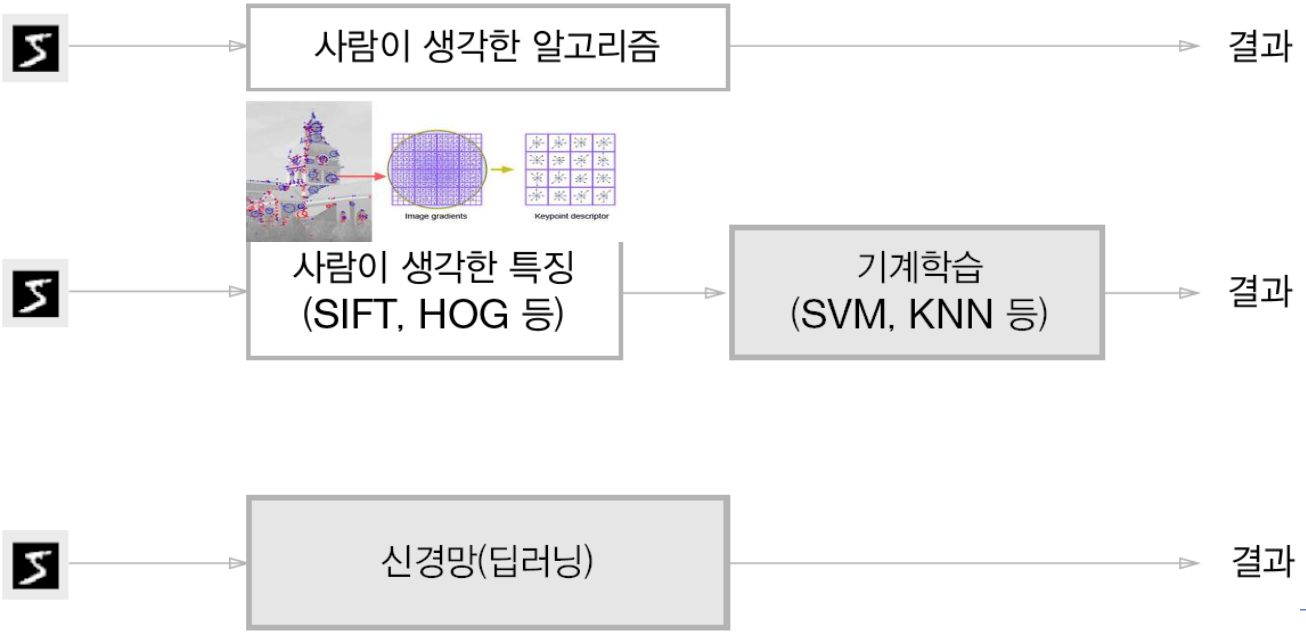
Neural Network Training

Neural Network Training

Data-driven learning

- How can we code a program that properly classifies the '5'?



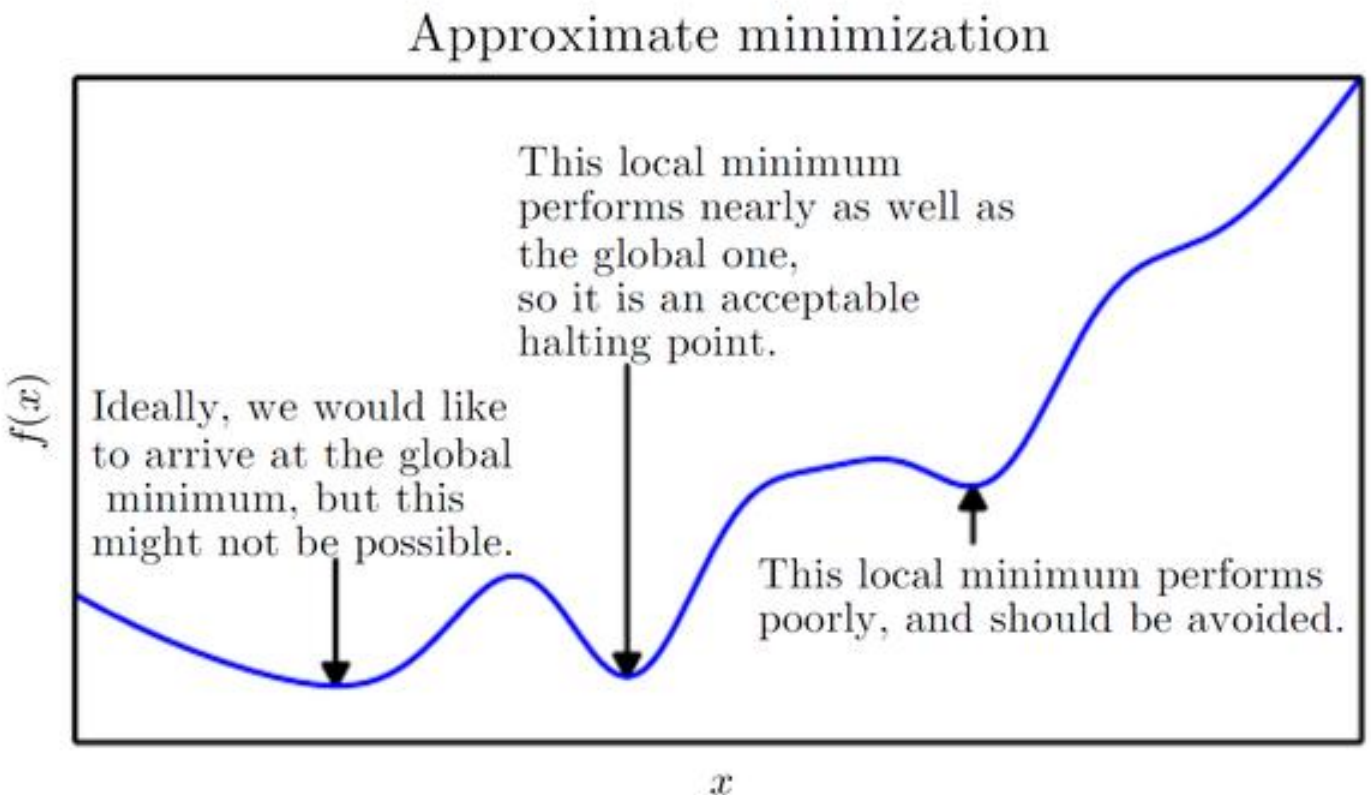


Neural Network Training

Loss function

- Indicates how poorly the current neural network is processing the training data (the gap between the predicted value and the actual value)
- The goal of neural network training is to minimize the loss function by optimizing parameters (parameters: weights, biases)

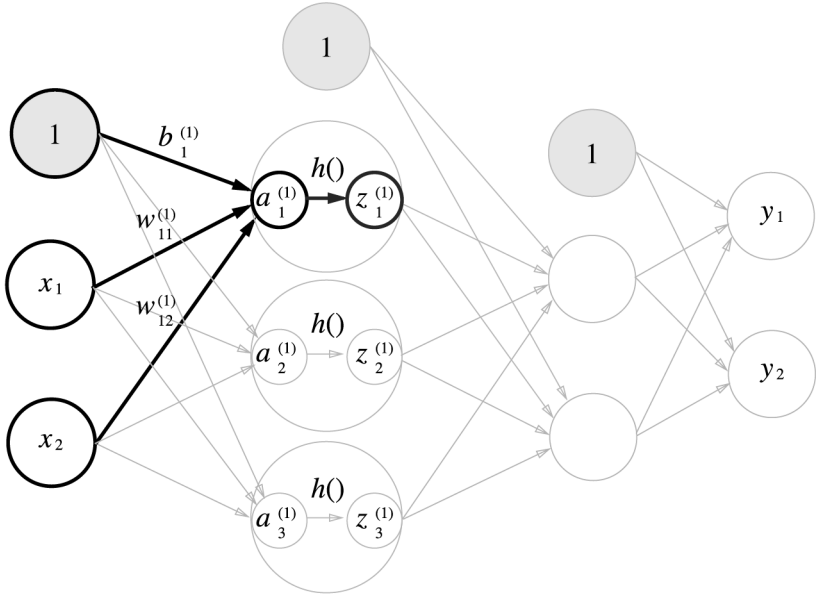
loss surface



Neural Network Training

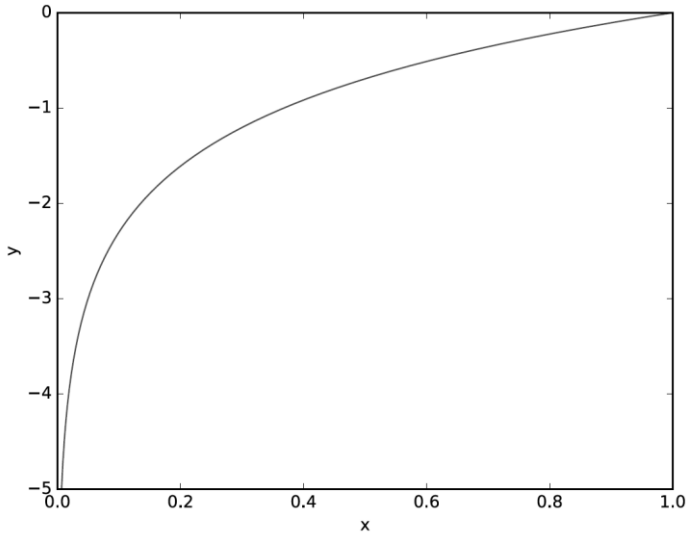
Loss function (1) SSE (Sum of Squares for Error)

$$E = \frac{1}{2}\sum_{k}(y_k - t_k)^2$$



Loss function (2) CEE (Cross Entropy Error)

$$E = -\sum_{k} t_k \log y_k$$

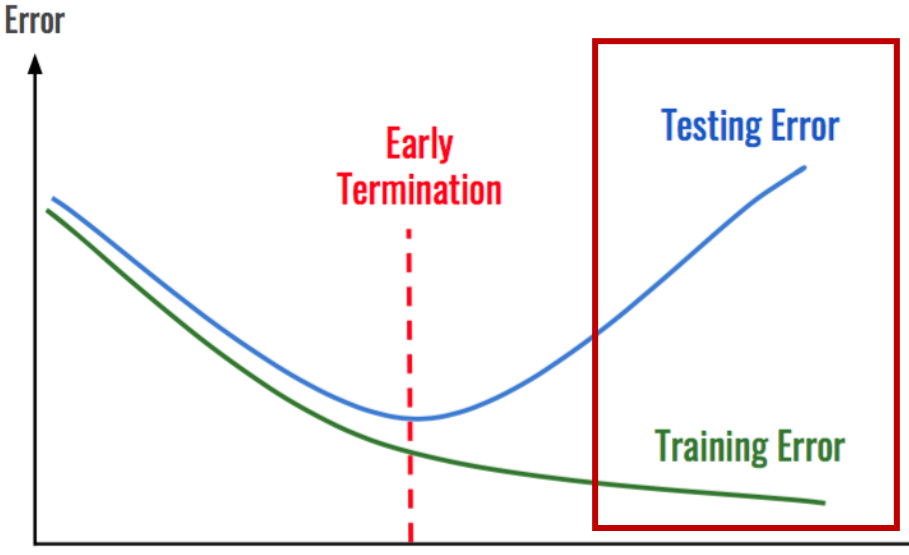


Neural Network Training

Training data & Test data

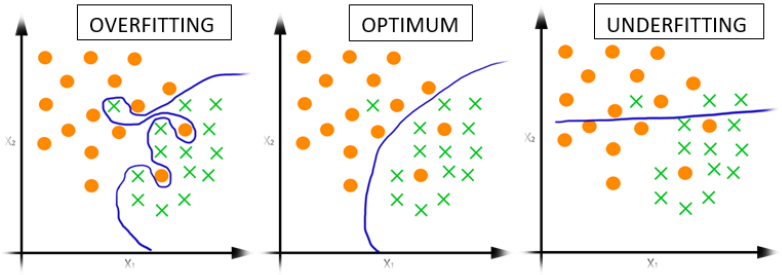
- First, find the optimal parameters by learning only the **training data**
- Then, evaluate the performance of the trained model through **test data**

Total data 100%		
Training Data 70%	**Validation Data** 15%	**Test Data** 15%



Overfitting

Models specific to training data and not universally available



early stopping => use validation

Neural Network Training

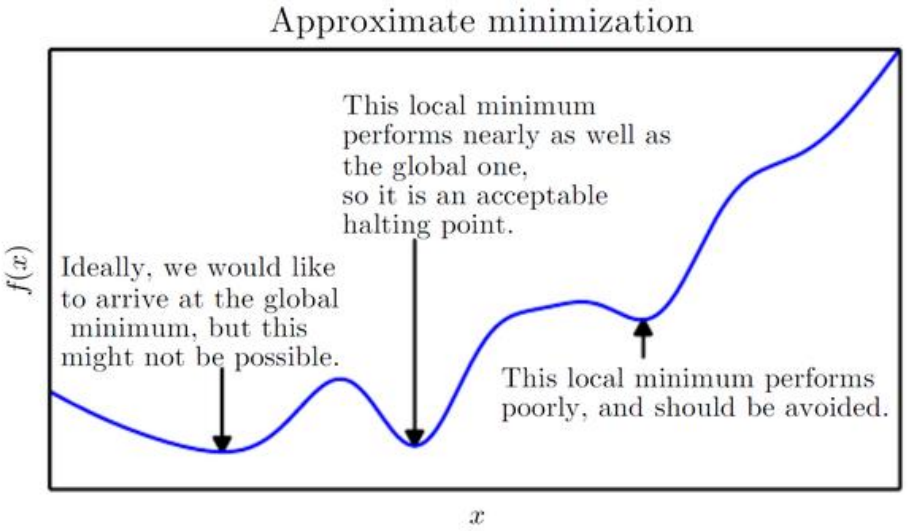
Mini batch & Epoch

- **Mini batch**: The neural network is trained by splitting the training data into batch sizes
- **1 iteration**: Learn batch size data once
- **1 Epoch**: Learn all training data once

- For example, if total training data = 100, batch size = 10,
  then 1 iteration = learn 10 data
  then 1 Epoch = 100/batch size = 10 iterations

Neural Network Training

Why mini batch size is better than one single "batch" with all training data?

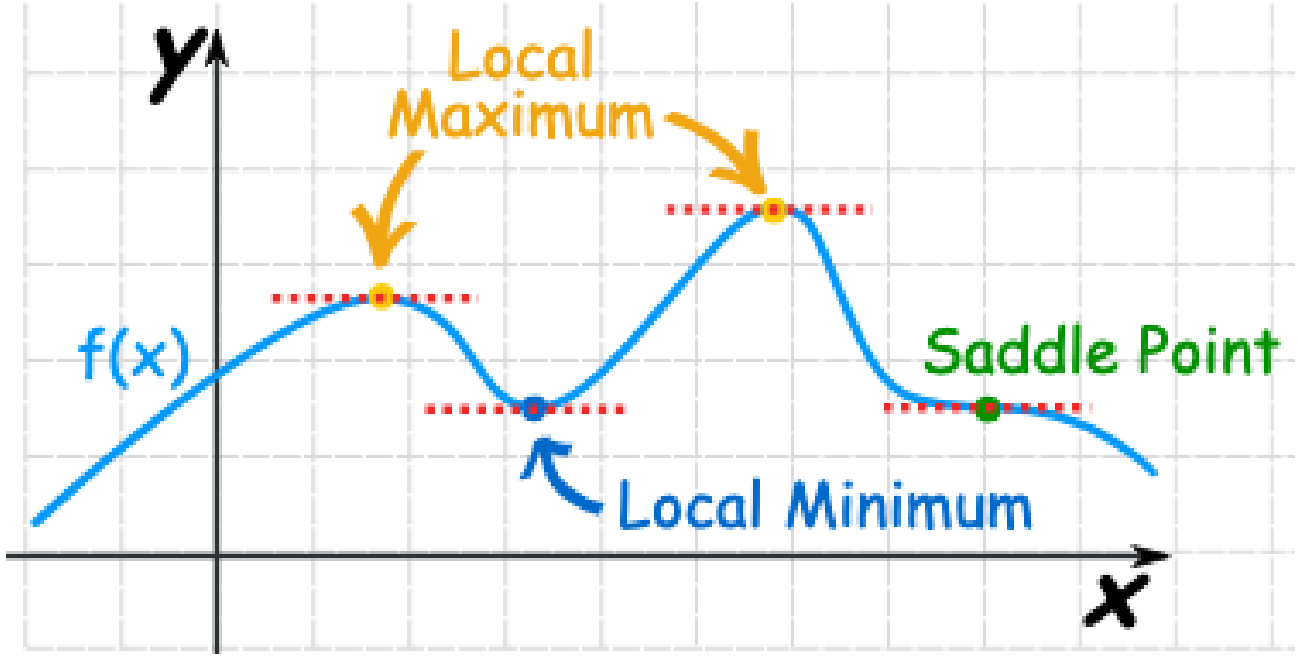


https://arxiv.org/abs/1804.07612

Neural Network Training

Points in the loss function

- Local maximum point: A point where f(x) is greater than all its neighbors
- Local minimum point: A point where f(x) is less than all its neighbors
- Saddle point: A critical point that is neither a minimum nor a maximum

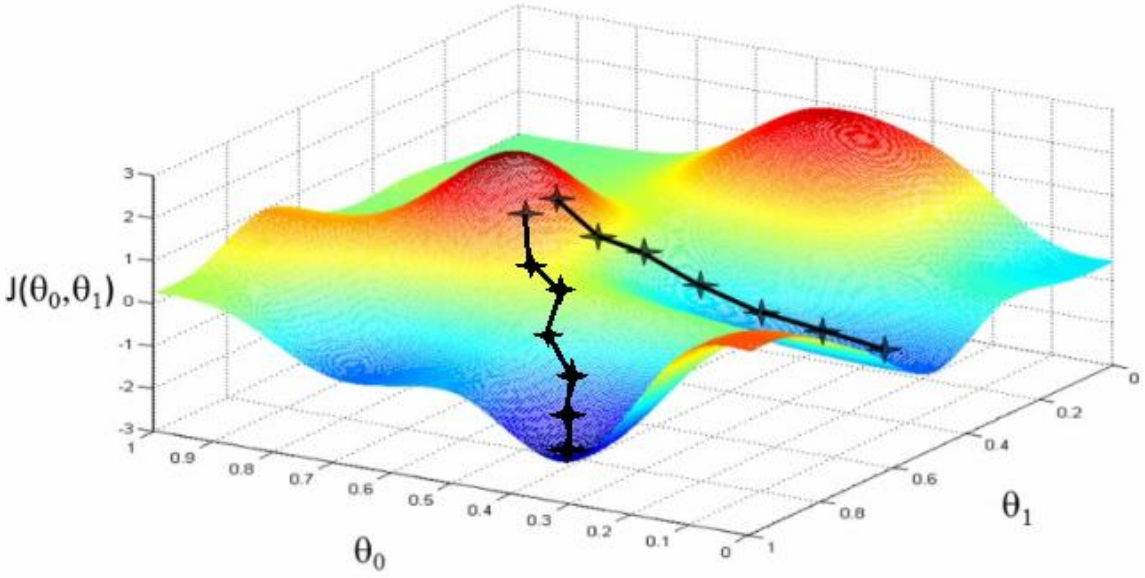


Neural Network Training Procedure

How to minimize the loss function by optimizing the parameters

- **Adjust the parameters** to minimize the loss function
- How? Use the **derivative** of the loss function of the weight parameter

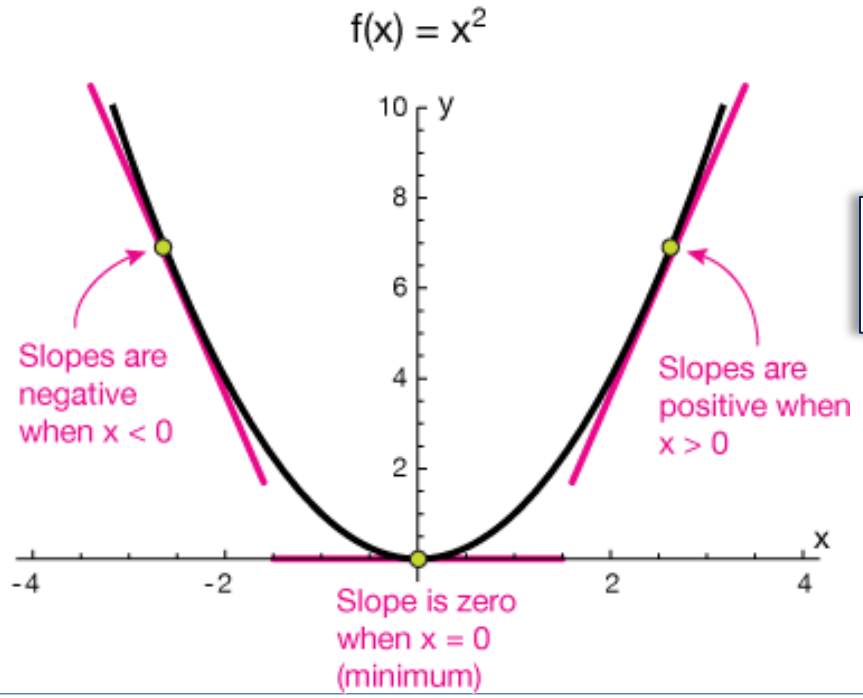
- Decides: 1) Which direction will you go down? & 2) How far will you travel at one time?



Neural Network Training Procedure

1) Which direction will you go down?

- Use the derivative of the one-dimensional loss function $f(x)$
- Derivative of loss function $f(x)$ $$\frac{df(x)}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$



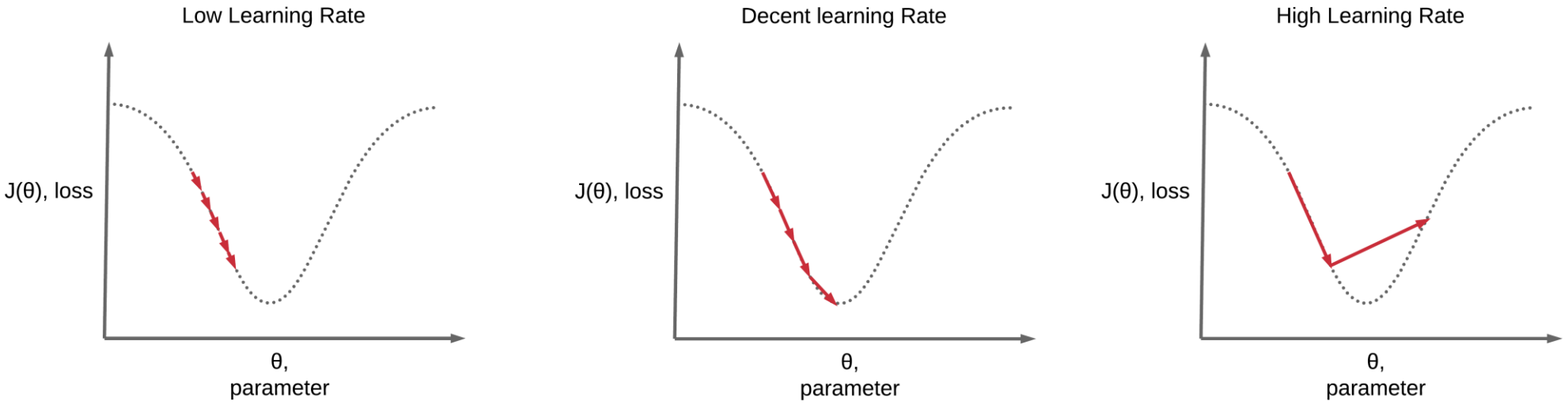
Decide which direction to go down through the derivative $f'(x)$

Neural Network Training Procedure

2) How far will you travel at one time?

- Low learning rate η → a lot of computation time
- High learning rate η → risk of skipping good values

- Learning rate = **Hyper parameter** (need to be set manually)



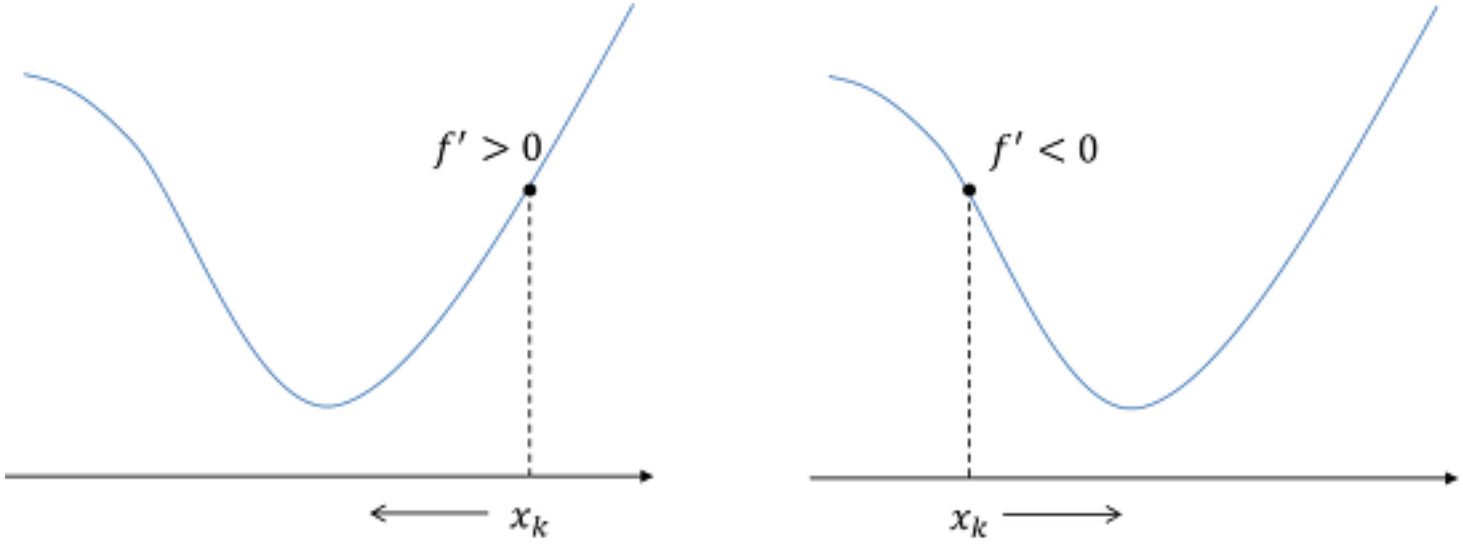
Neural Network Training Procedure

Optimization with Differentiation

$$x_{k+1} = x_k - \eta f'(x_k)$$

- Starting from point x_k, if the first derivative is positive ($f'(x_k) > 0$) then decrease x_{k+1}, if negative ($f'(x_k) < 0$) then increase x_{k+1}→ Find the point where $f(x_k)$ is a minimum

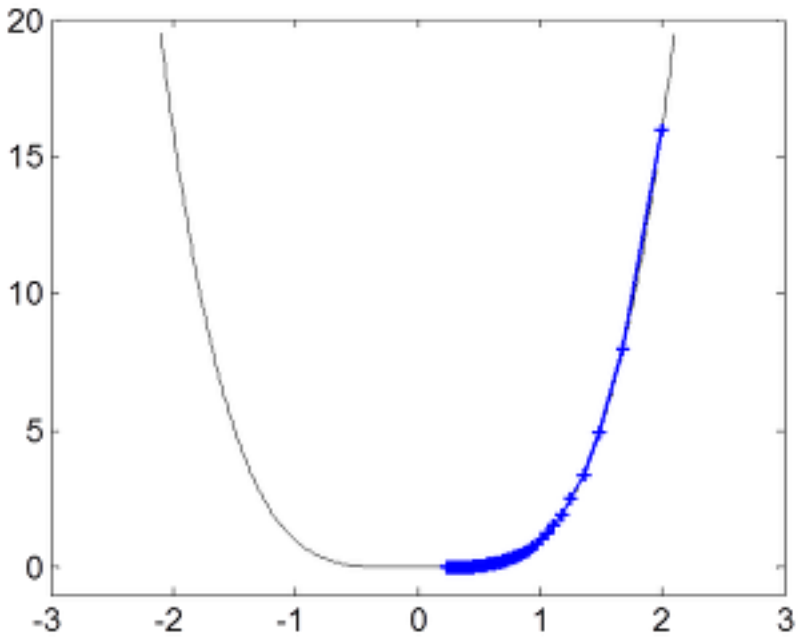
- η is the learning rate of how far to move at a time



Neural Network Training Procedure

Optimization with Differentiation (example)

- Minimize $f(x) = x^4$
- Start from $x_0 = 2$
- Learning rate $\eta = 0.01$ (200 iterations)

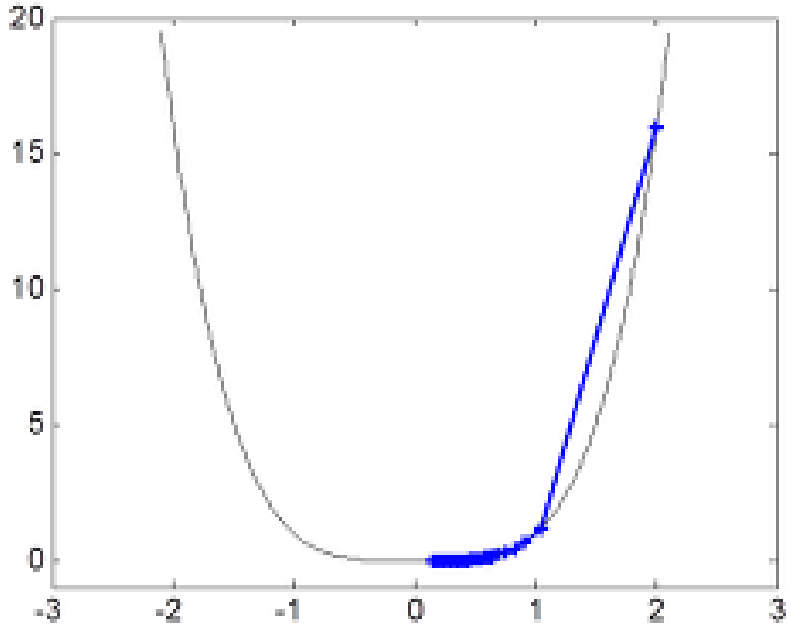


$x_0 = 2.0$
$x_1 = x_0 - \lambda f'(x_0) = 1.6800$
$x_2 = x_1 - \lambda f'(x_1) = 1.4903$
$x_3 = x_2 - \lambda f'(x_2) = 1.3579$
$x_4 = x_3 - \lambda f'(x_3) = 1.2578$
$x_5 = x_4 - \lambda f'(x_4) = 1.1782$
$x_6 = x_5 - \lambda f'(x_5) = 1.1128$
$x_7 = x_6 - \lambda f'(x_6) = 1.0576$
$x_8 = x_7 - \lambda f'(x_7) = 1.0103$
$x_9 = x_8 - \lambda f'(x_8) = 0.9691$
...

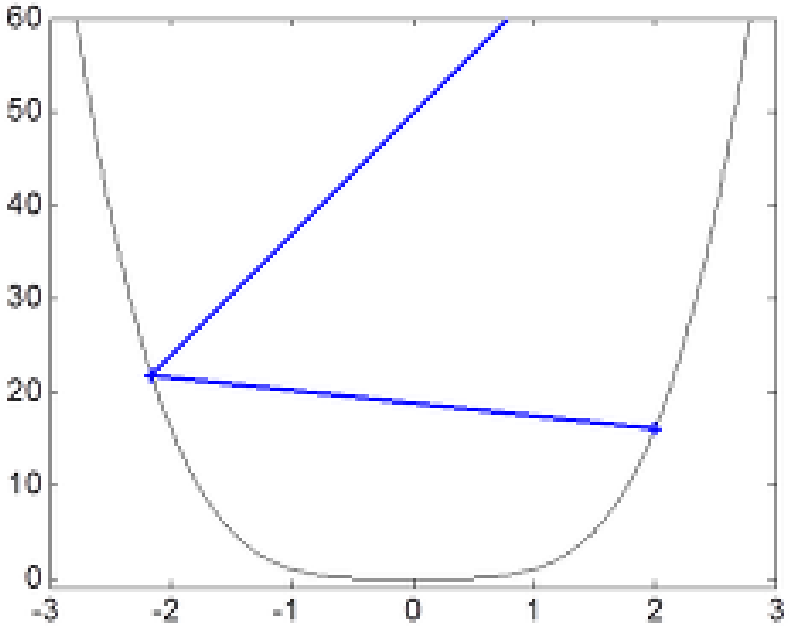
Neural Network Training Procedure

Optimization with Differentiation (example)

- When learning rate $\eta = 0.03$ (3x increase)
- When learning rate $\eta = 0.13$ (13x increase)



$\lambda = 0.03$



$\lambda = 0.13$

Neural Network Training Procedure

Optimization in multivariate functions

- Multivariable function: A function whose value is determined by two or more variables (Example: $f(x_0, x_1) = x_0^2 + x_0 x_1 + x_1^2$)

- **Gradient**: Calculate the partial derivative of the $f(x_0, x_1)$ function for each variable x_0, x_1

$$x_0 = x_0 - \eta \frac{\partial f}{\partial x_0}$$

$$x_1 = x_1 - \eta \frac{\partial f}{\partial x_1}$$

→ **Gradient descent method**

Neural Network Training Procedure

Using Gradient Descent Method

1) Mini batch: Some of the training data are taken randomly and used for training

2) Gradient calculation: Calculate the gradient of each weight parameter to reduce the loss function value of the mini-batch

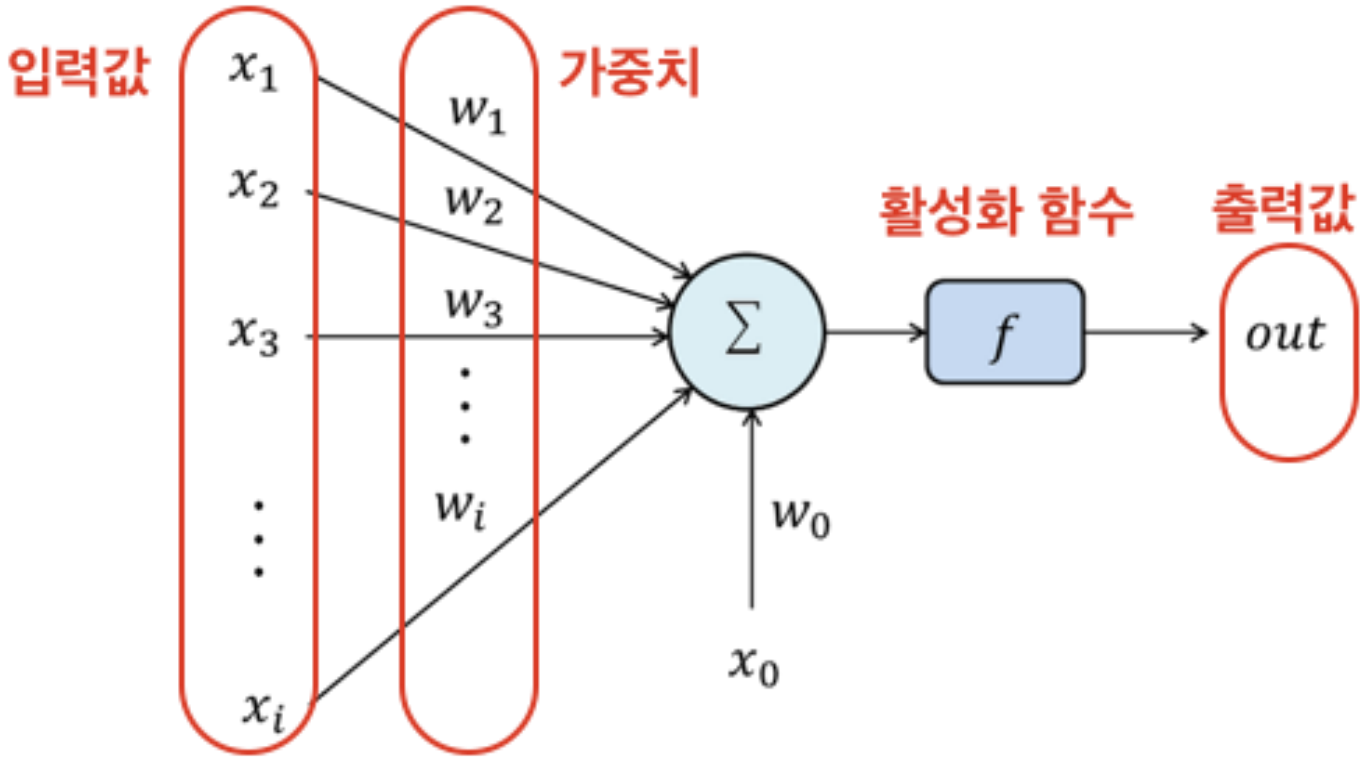
3) Parameter update: Update the weight parameter in the direction of the gradient (the gradient gives the direction of the smallest value of the loss function)

4) Repeat steps 1)~3)

Error Back Propagation

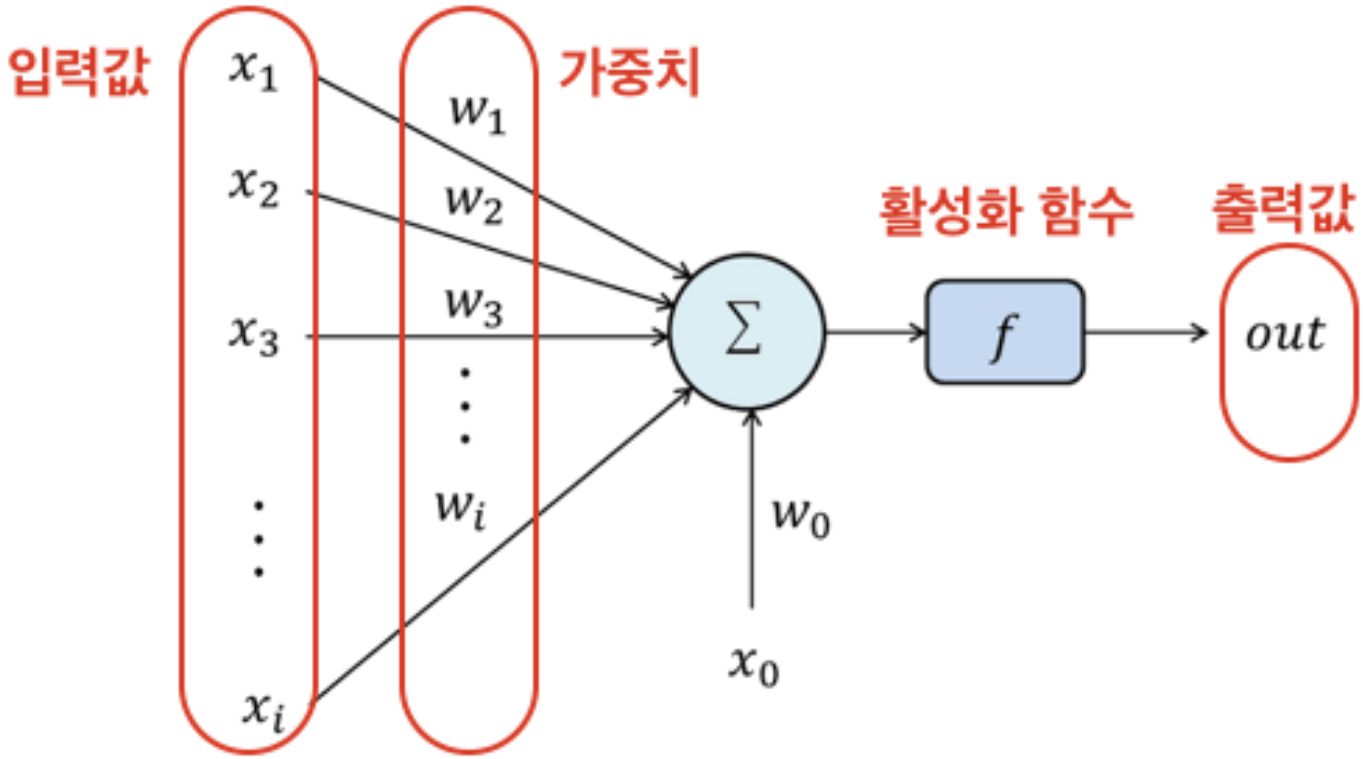
Error Back Propagation

- **Feedforward:** Propagation from the input layer to the output layer
- **Back propagation:** Update the weights by sending error from the output to the input layer



Error Back Propagation

- **Numerical differentiation:** Simple & easy to implement, but takes a long time to calculate
- **Error back propagation:** Efficiently compute gradients of weight parameters using chain rule

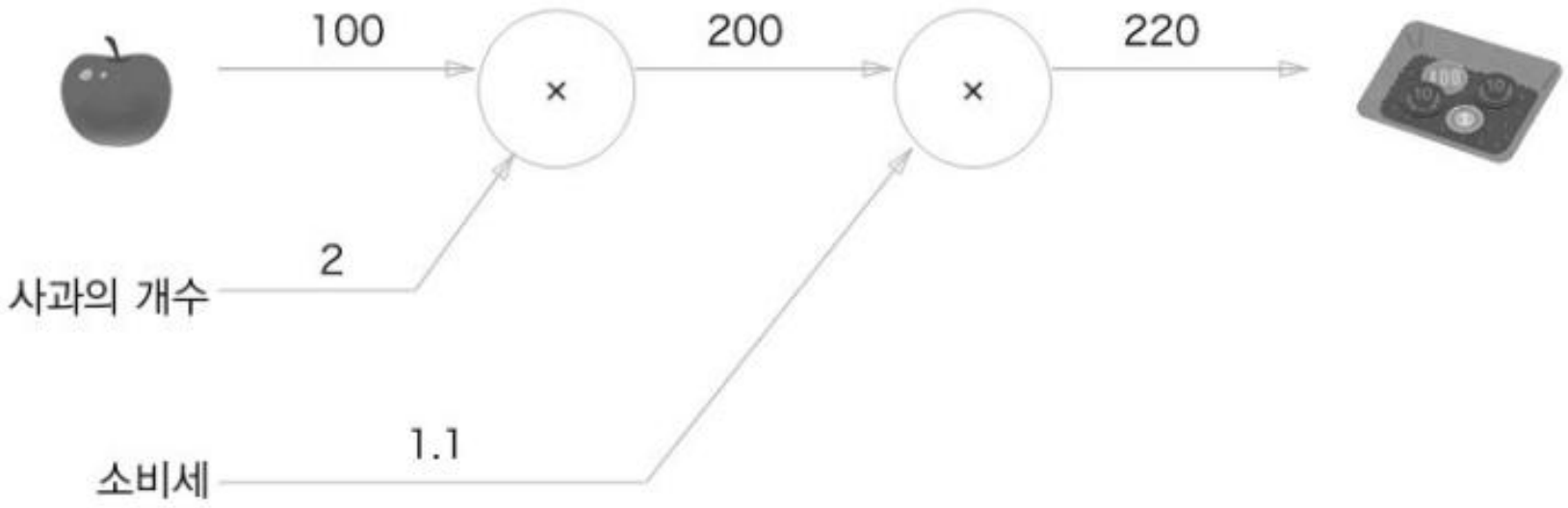


Error Back Propagation

Computational Graph

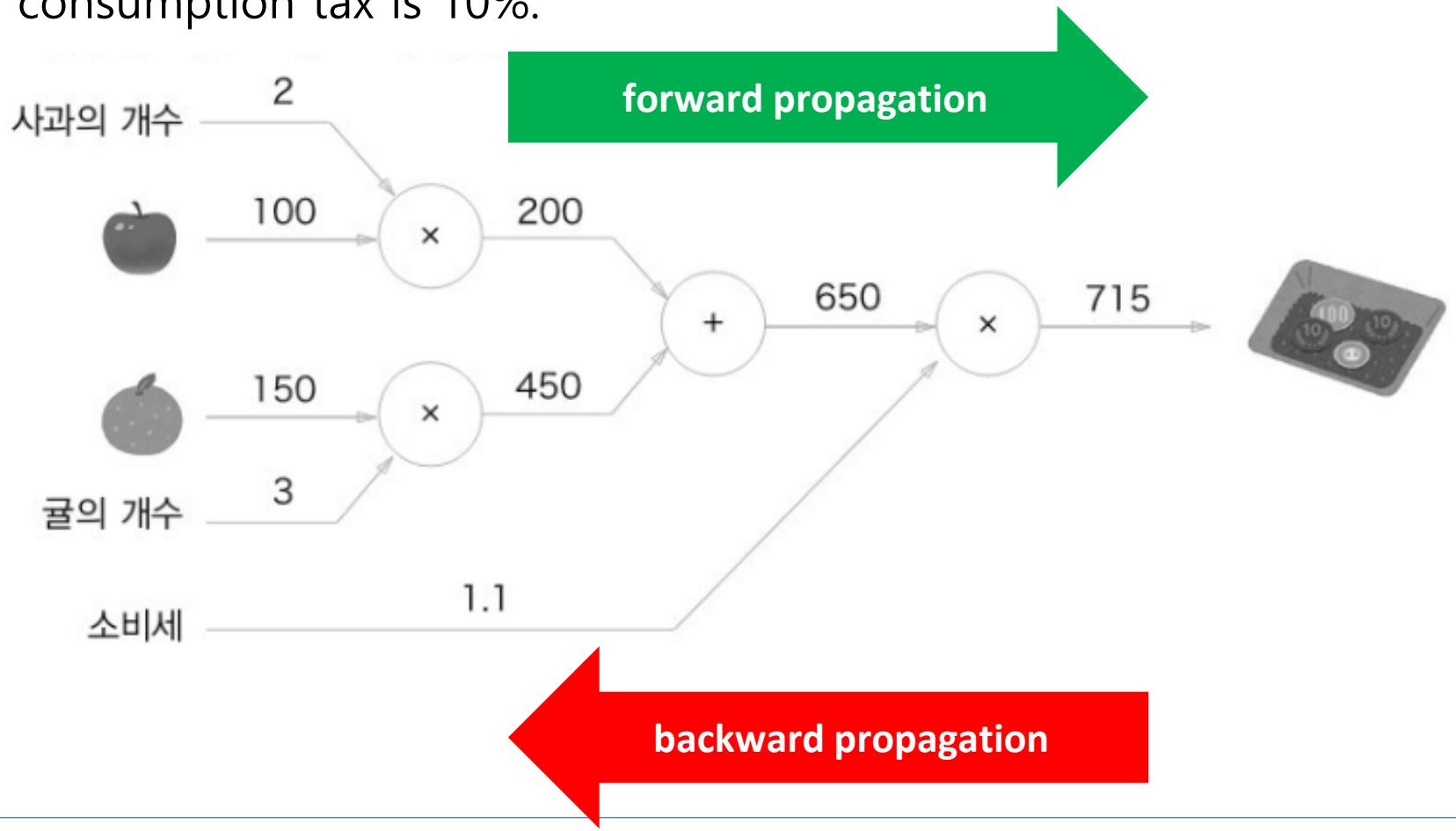
- A graphical representation of a computational process
- Represented by multiple nodes and edges

- **Example 1:** Donggyu bought two apples,100 won each at the supermarket. Calculate the amount to be paid when a consumption tax of 10% is charged.



Error Back Propagation

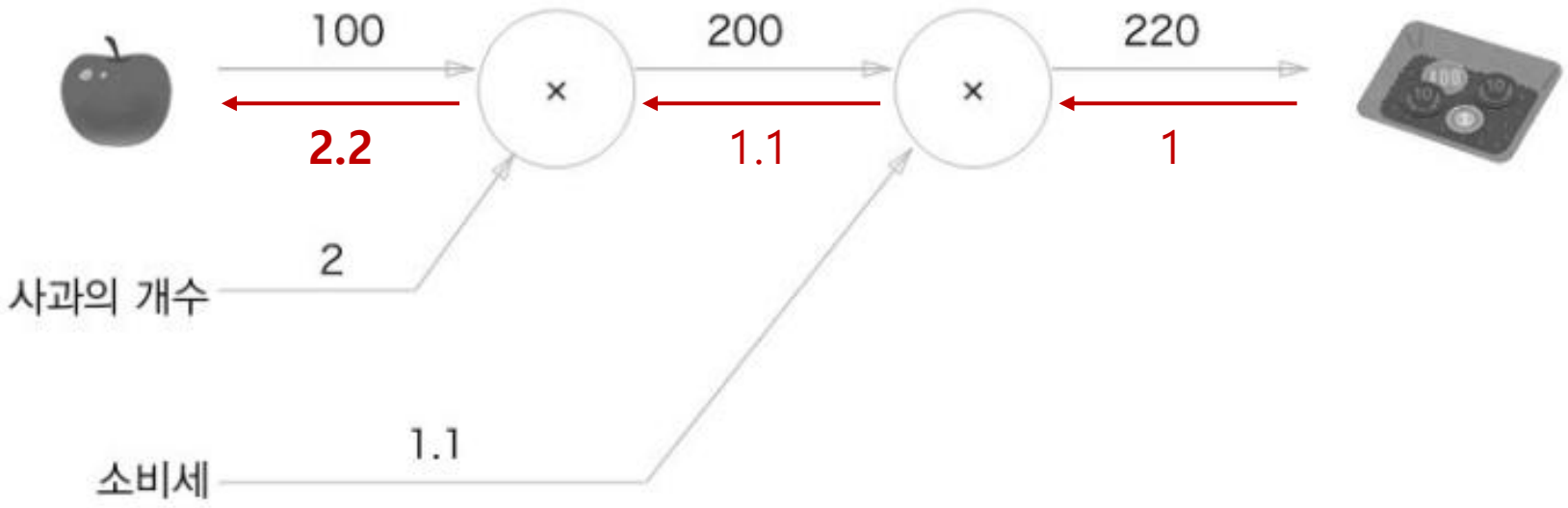
- **Example 2:** Yun Ji bought 2 apples and 3 tangerines at the supermarket. Apples cost 100 won per piece and tangerines cost 150 won per piece. Calculate the amount to be paid when the consumption tax is 10%.



Error Back Propagation

Why Use Calculation Graphs?

- Local calculation
- Archive intermediate calculation results
- Calculating the 'derivative' via backpropagation
  (e.g., how does an increase in the price of an apple affect the final price?)



If the apple price rises, the final amount increases by 2.2 times.

Error Back Propagation

Chain Rule

- Numerical differentiation is simple and easy to implement, but takes a long time to calculate

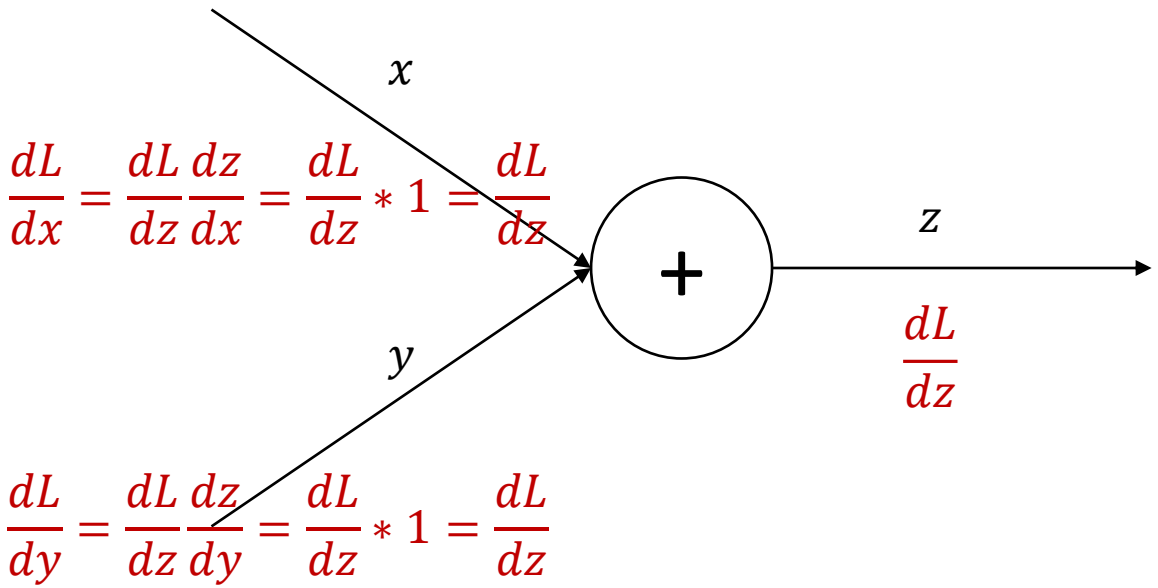
- Composite function: a function made up of multiple functions
  (e.g., $z = (x + y)^2$ is composed of two expressions $z = t^2,\ t = x + y$)

- The chain rule can express the derivative of a composite function as the product of the derivatives of each function constituting the composite function

→ $\frac{dz}{dx} = \frac{dz}{dt}\frac{dt}{dx} = (2t)(1) = 2(x + y)$

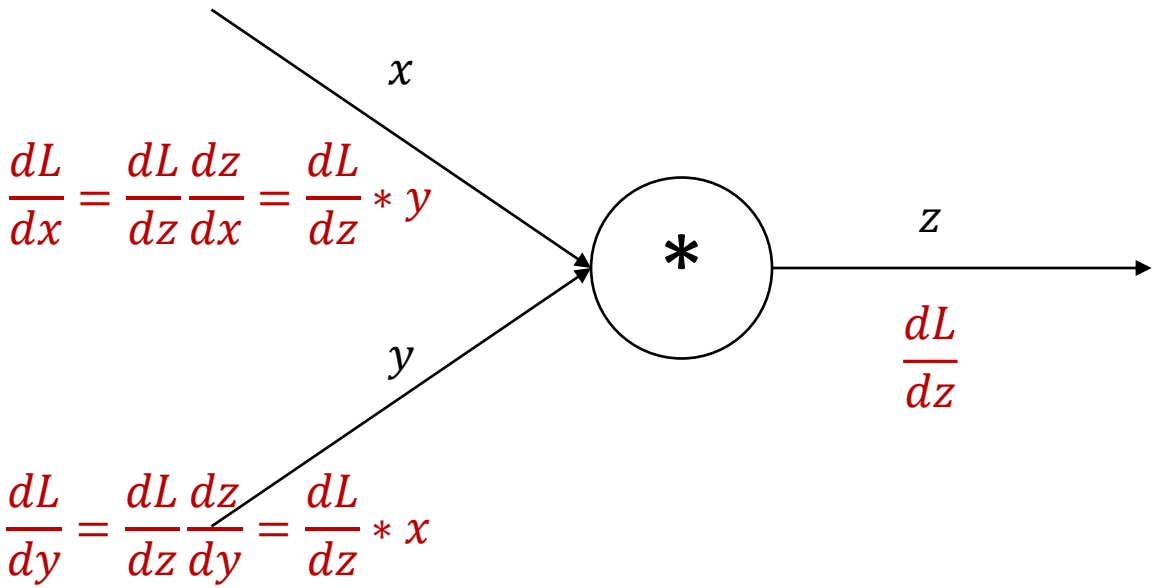
Error Back Propagation

Addition Node: Backpropagating the expression $z=x+y$

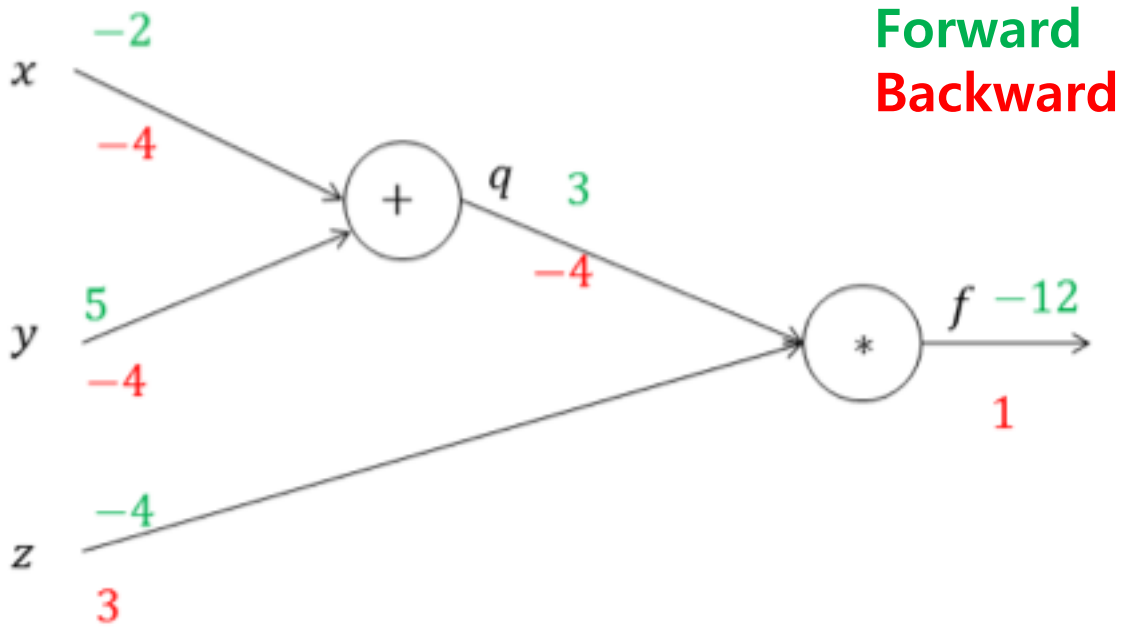


Error Back Propagation

Multiplication Node: Backpropagating the expression $z=xy$



Error Back Propagation (Example 1)



Forward
Backward

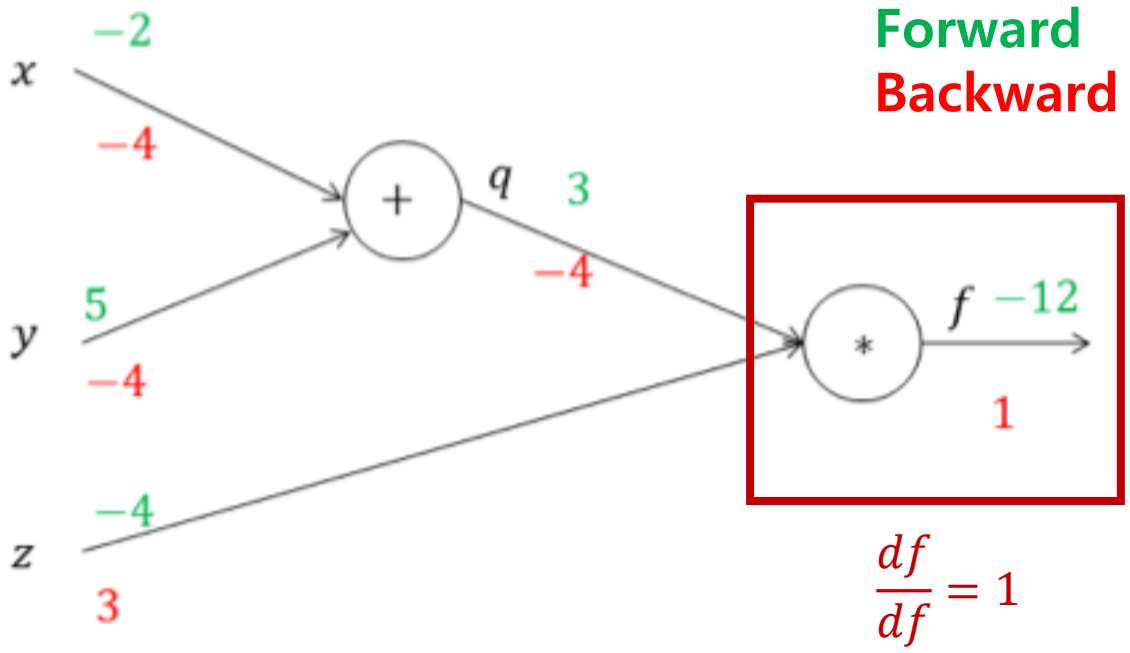
$$f(x, y, z) = (x + y)z$$

$$x = -2, \qquad y = 5, \qquad z = -4$$

$$q = (x + y)$$

$$f = qz$$

Error Back Propagation (Example 1)



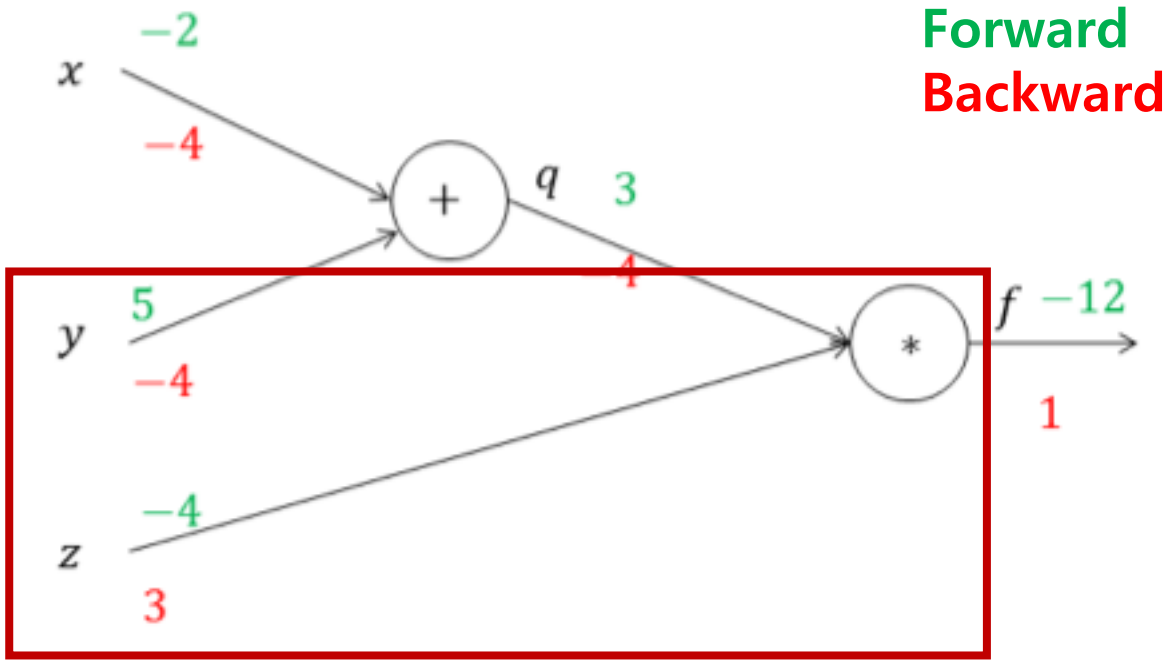
$$f(x, y, z) = (x + y)z$$

$$x = -2, \qquad y = 5, \qquad z = -4$$

$$q = (x + y)$$

$$f = qz$$

Error Back Propagation (Example 1)



Forward
Backward

$$f(x, y, z) = (x + y)z$$

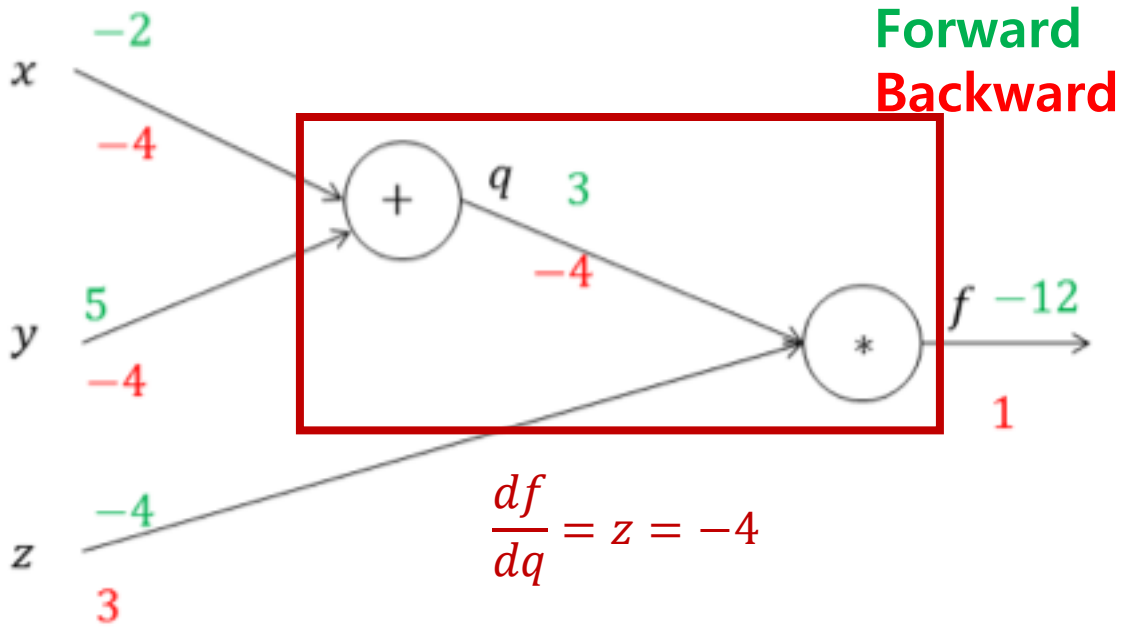
$$x = -2, \qquad y = 5, \qquad z = -4$$

$$q = (x + y)$$

$$f = qz$$

$$\frac{df}{dz} = q = x + y = 3$$

Error Back Propagation (Example 1)



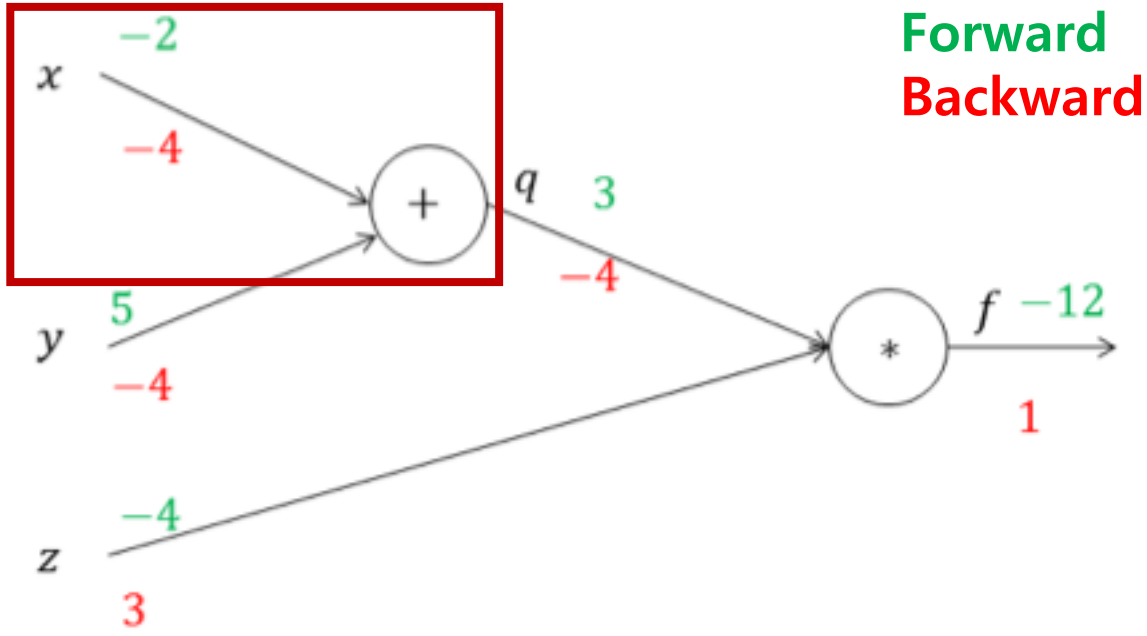
$$f(x, y, z) = (x + y)z$$

$$x = -2, \qquad y = 5, \qquad z = -4$$

$$q = (x + y)$$

$$f = qz$$

Error Back Propagation (Example 1)



Forward
Backward

$$f(x, y, z) = (x + y)z$$

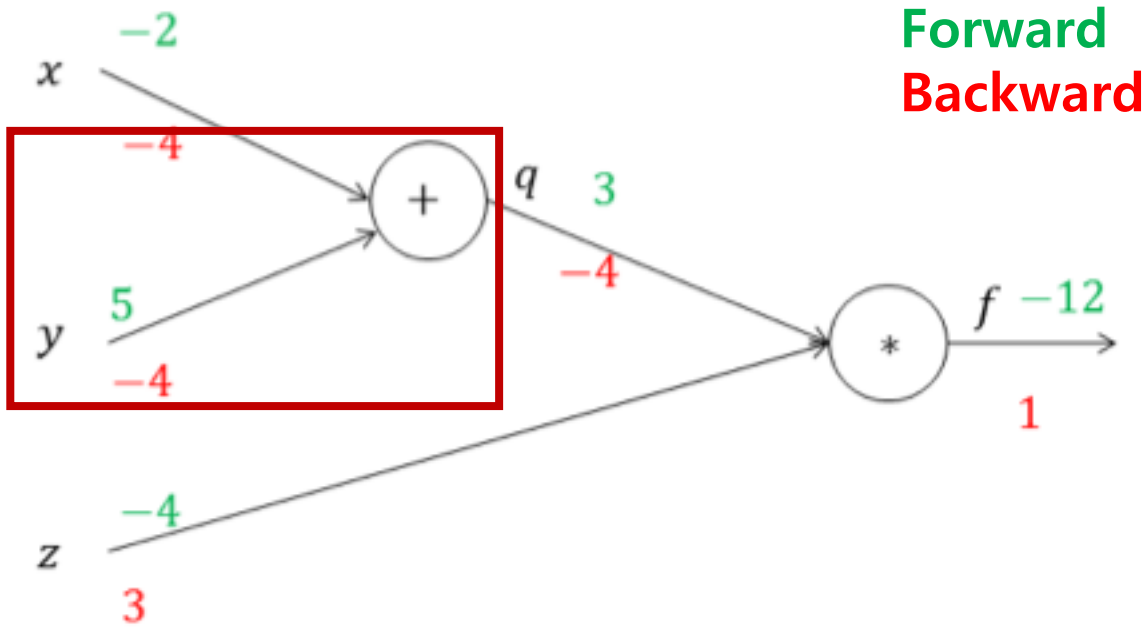
$$x = -2, \qquad y = 5, \qquad z = -4$$

$$q = (x + y)$$

$$f = qz$$

$$\frac{df}{dx} = \frac{df}{dq}\frac{dq}{dx} = (-4)(1) = -4$$

Error Back Propagation (Example 1)



Forward
Backward

$$f(x, y, z) = (x + y)z$$

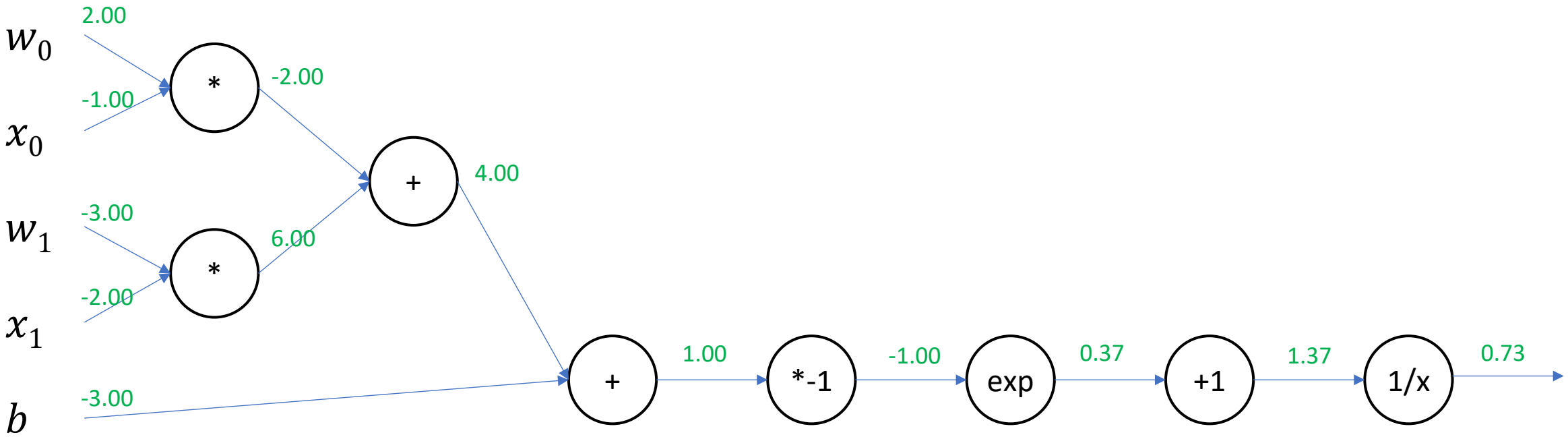
$$x = -2, \qquad y = 5, \qquad z = -4$$

$$q = (x + y)$$

$$f = qz$$

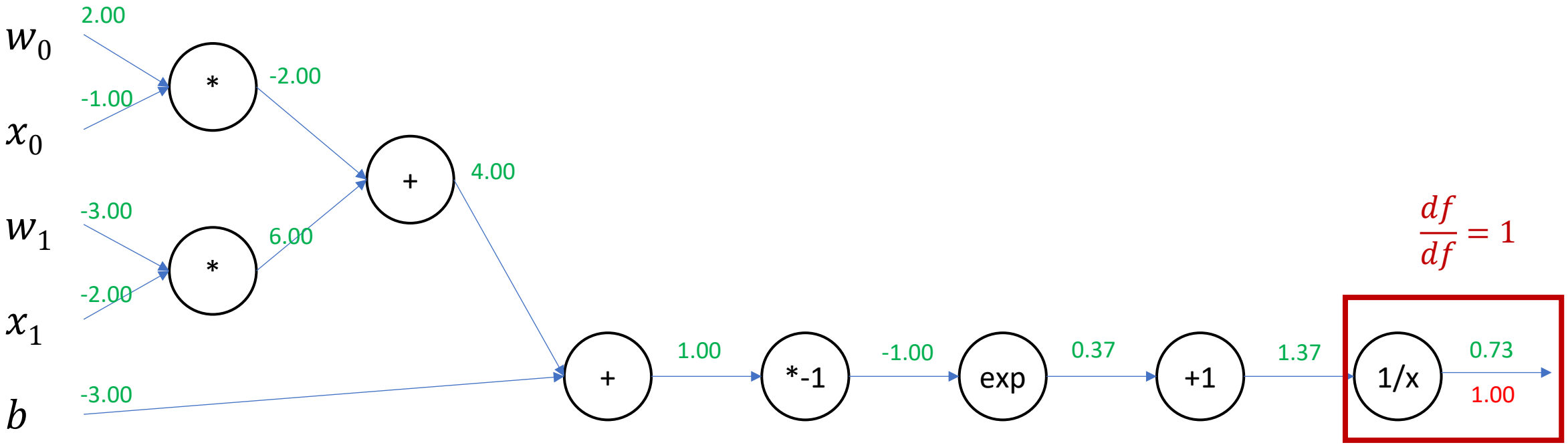
$$\frac{df}{dy} = \frac{df}{dq}\frac{dq}{dy} = (-4)(1) = -4$$

Error Back Propagation (Example 2)



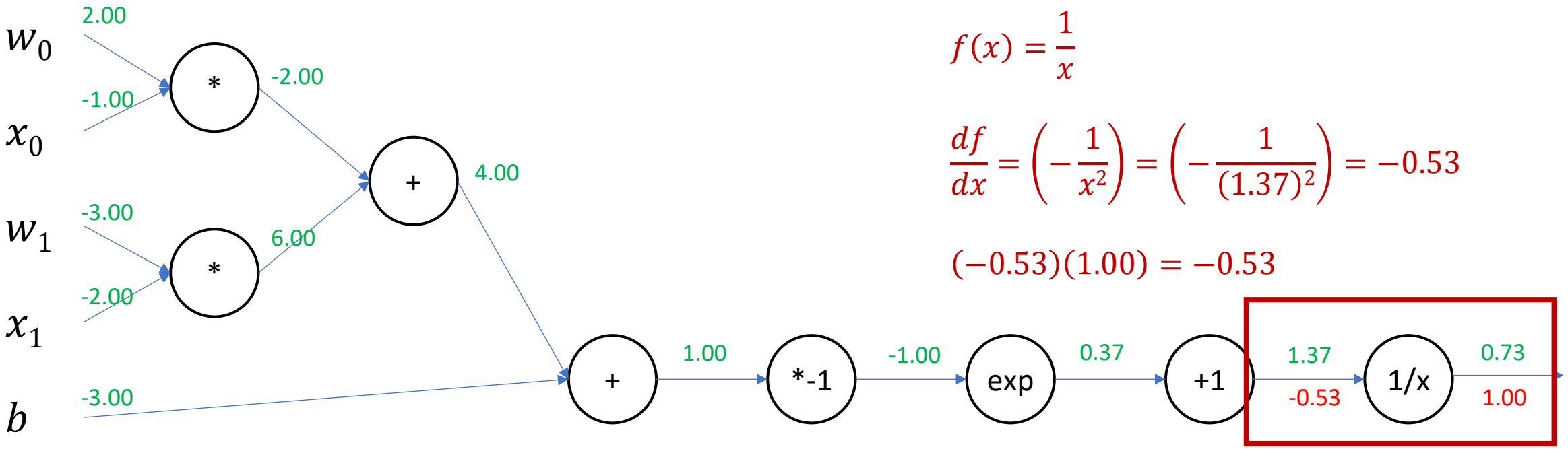
$$f(w,x) = \frac{1}{1 + e^{-(b+w_0x_0+w_1x_1)}}$$

Error Back Propagation (Example 2)



$$f(w,x) = \frac{1}{1 + e^{-(b+w_0x_0+w_1x_1)}}$$

Error Back Propagation (Example 2)



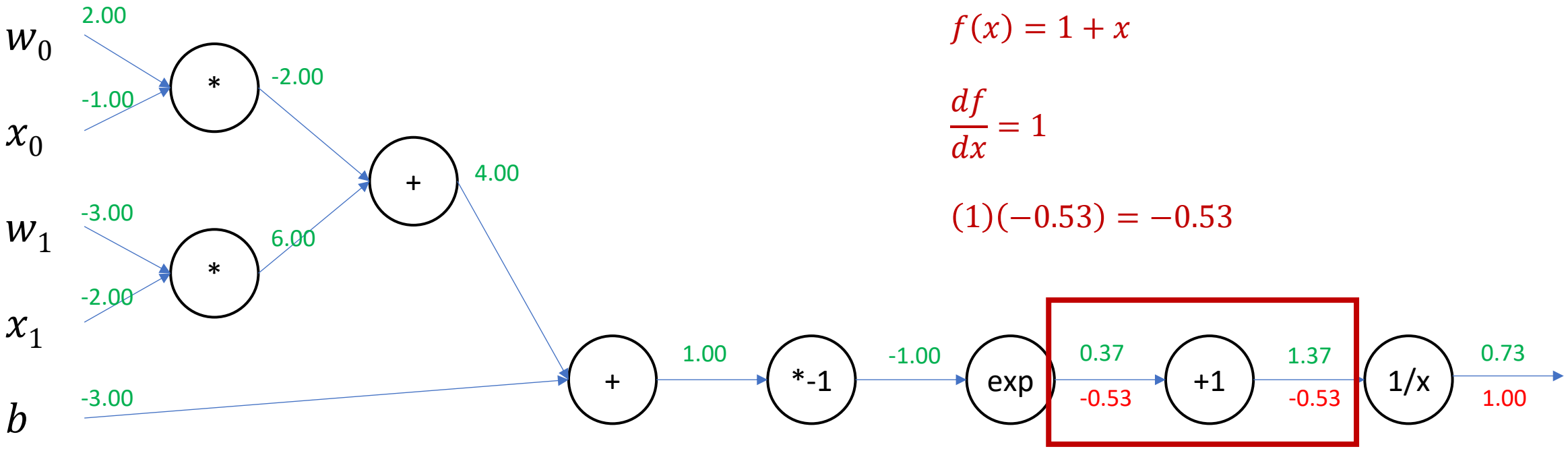
$$f(x) = \frac{1}{x}$$

$$\frac{df}{dx} = \left(-\frac{1}{x^2}\right) = \left(-\frac{1}{(1.37)^2}\right) = -0.53$$

$$(-0.53)(1.00) = -0.53$$

$$f(w, x) = \frac{1}{1 + e^{-(b + w_0 x_0 + w_1 x_1)}}$$

Error Back Propagation (Example 2)



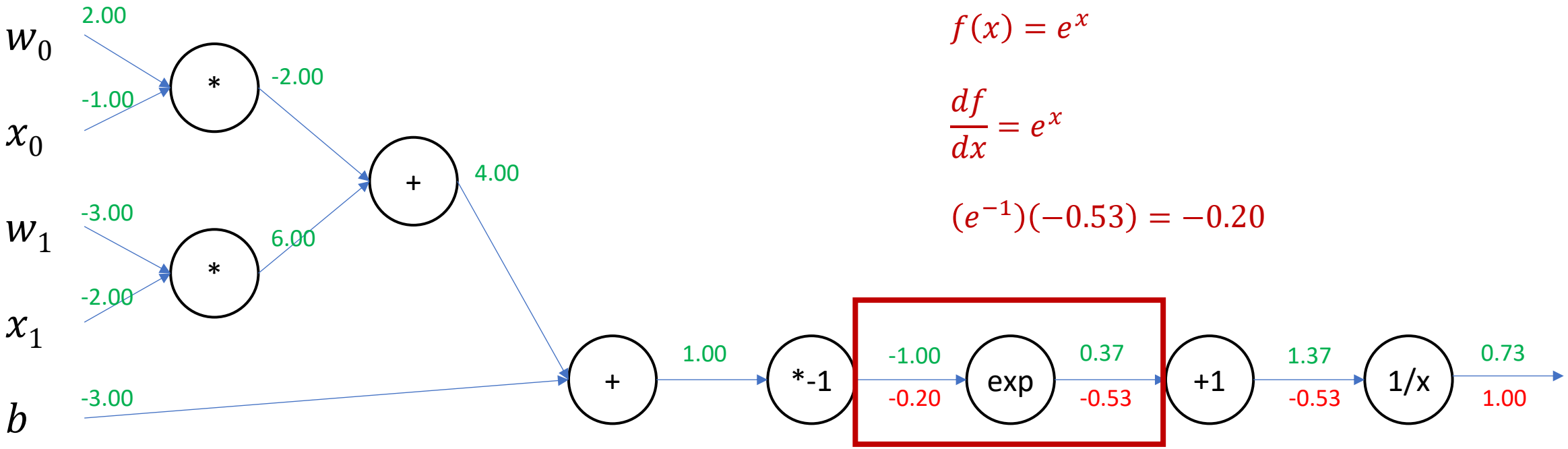
$$f(x) = 1 + x$$

$$\frac{df}{dx} = 1$$

$$(1)(-0.53) = -0.53$$

$$f(w, x) = \frac{1}{1 + e^{-(b + w_0 x_0 + w_1 x_1)}}$$

Error Back Propagation (Example 2)



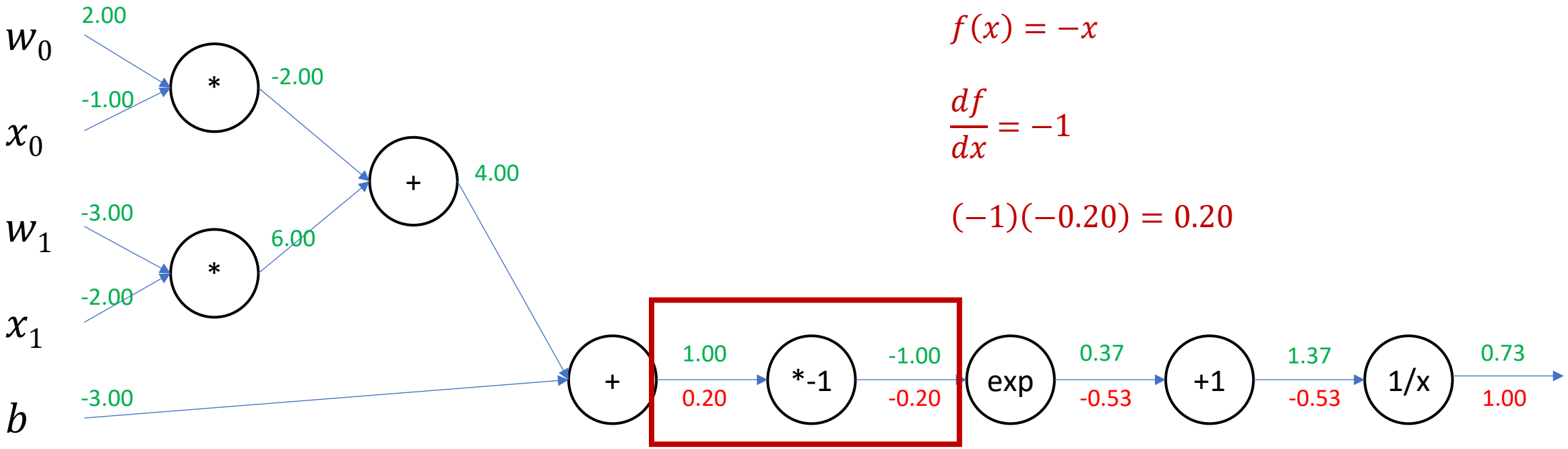
$$f(x) = e^x$$

$$\frac{df}{dx} = e^x$$

$$(e^{-1})(-0.53) = -0.20$$

$$f(w,x) = \frac{1}{1 + e^{-(b+w_0x_0+w_1x_1)}}$$

Error Back Propagation (Example 2)



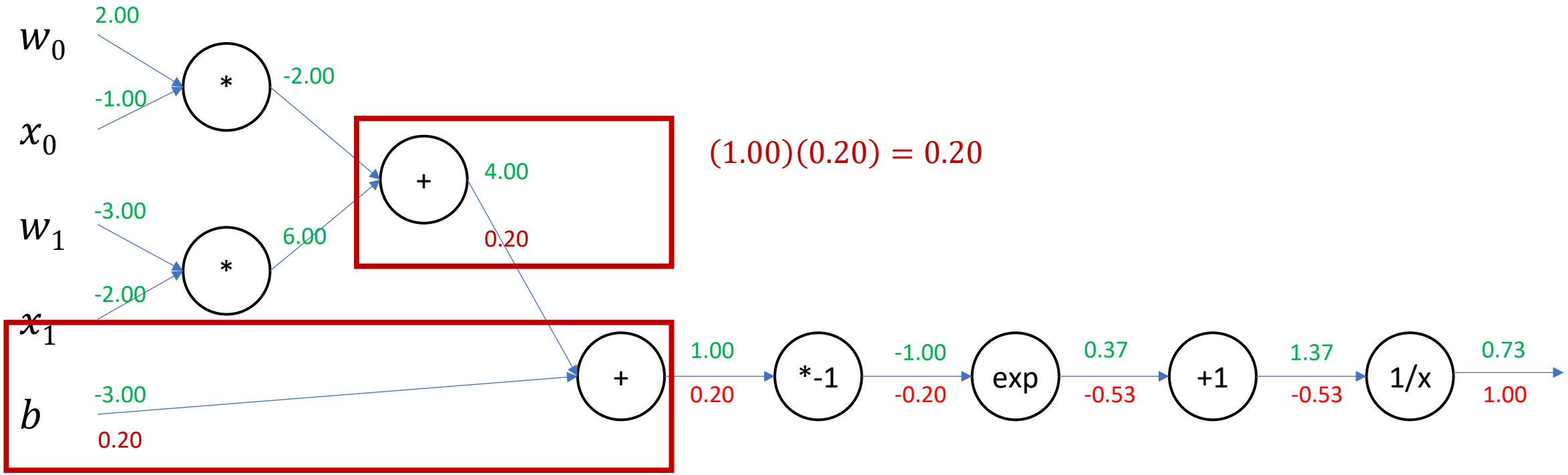
$$f(x) = -x$$

$$\frac{df}{dx} = -1$$

$$(-1)(-0.20) = 0.20$$

$$f(w,x) = \frac{1}{1 + e^{-(b+w_0x_0+w_1x_1)}}$$

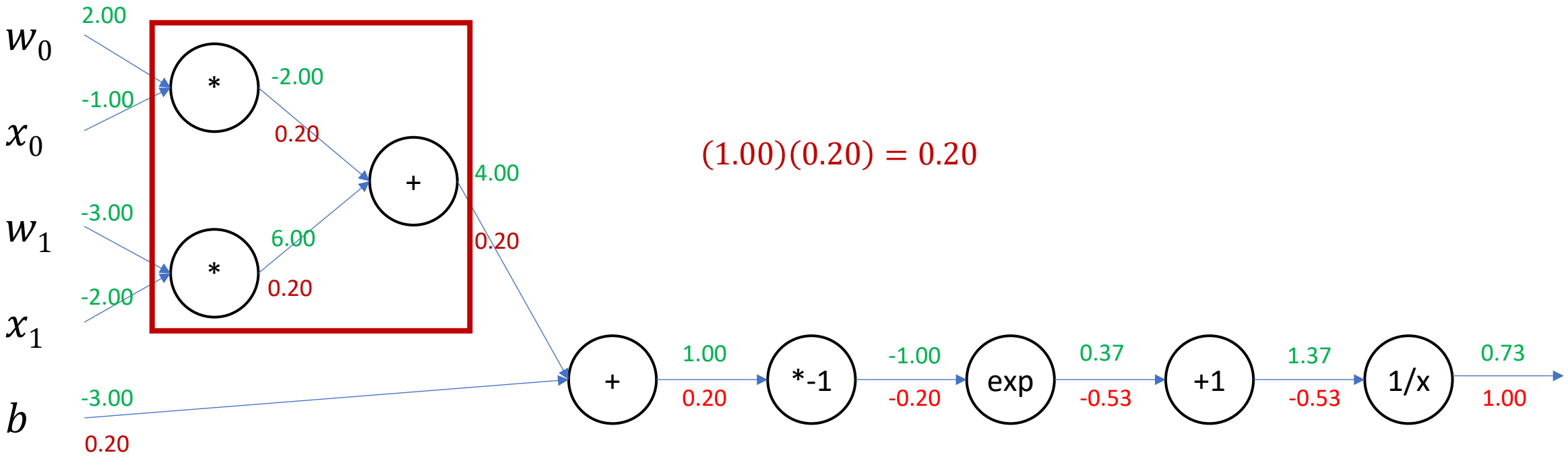
Error Back Propagation (Example 2)



$$(1.00)(0.20) = 0.20$$

$$f(w,x) = \frac{1}{1 + e^{-(b+w_0x_0+w_1x_1)}}$$

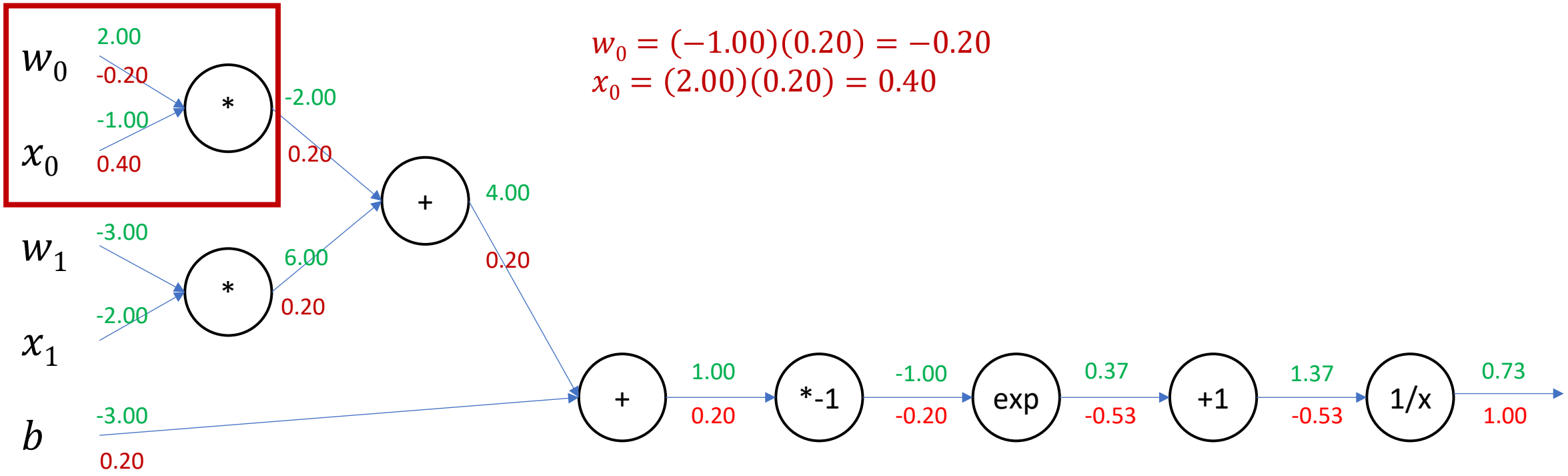
Error Back Propagation (Example 2)



$$(1.00)(0.20) = 0.20$$

$$f(w,x) = \frac{1}{1 + e^{-(b+w_0x_0+w_1x_1)}}$$

Error Back Propagation (Example 2)

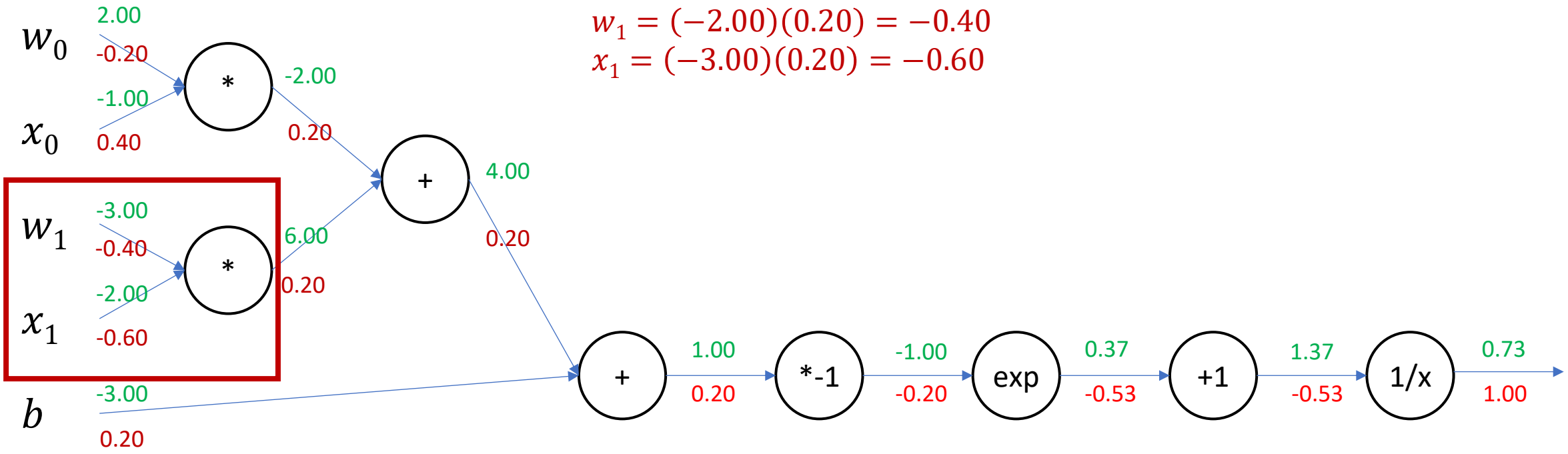


$$w_0 = (-1.00)(0.20) = -0.20$$
$$x_0 = (2.00)(0.20) = 0.40$$

$$f(w,x) = \frac{1}{1 + e^{-(b + w_0 x_0 + w_1 x_1)}}$$

Error Back Propagation (Example 2)

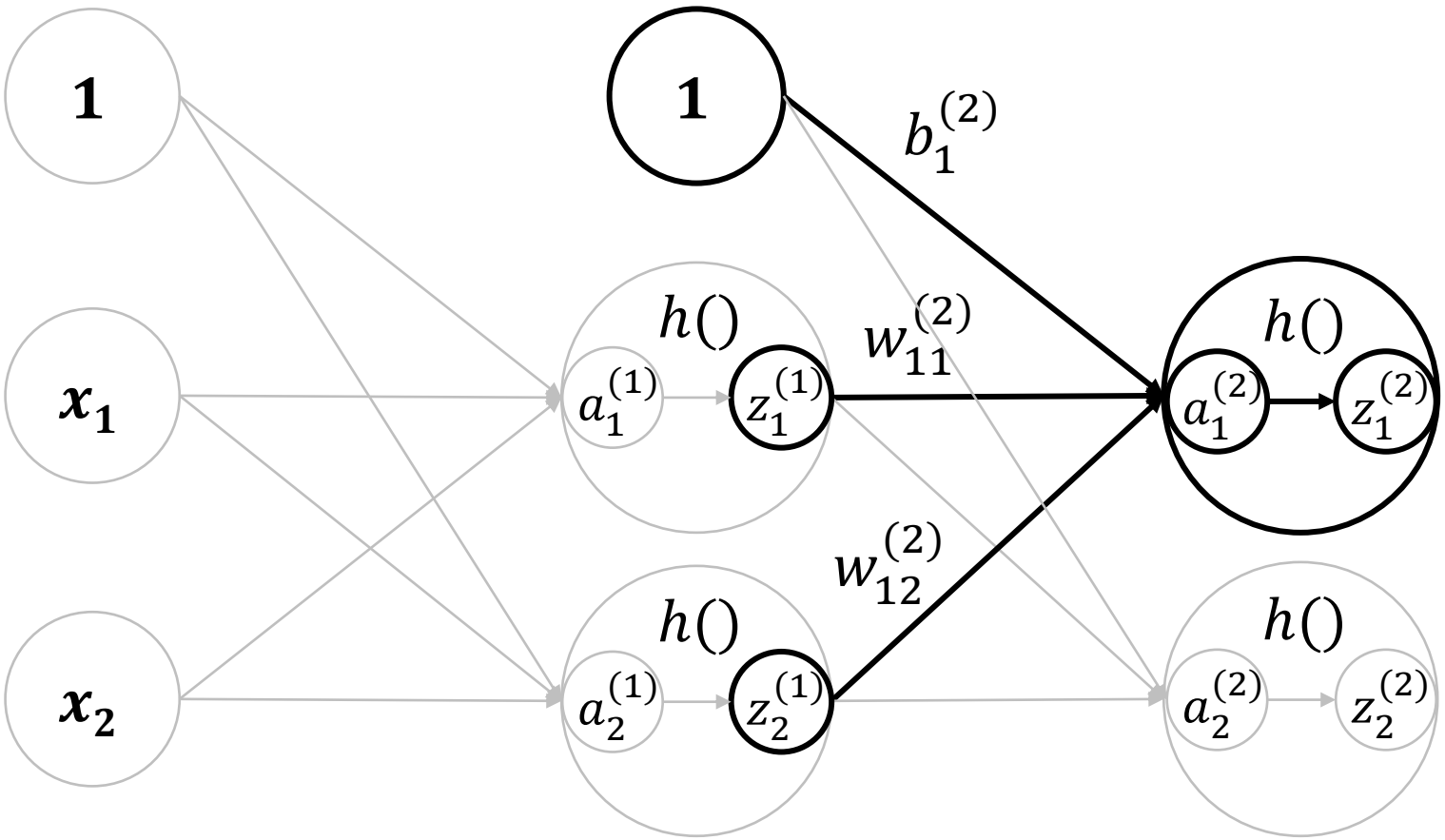
$w_1 = (-2.00)(0.20) = -0.40$
$x_1 = (-3.00)(0.20) = -0.60$



$$f(w,x) = \frac{1}{1 + e^{-(b + w_0 x_0 + w_1 x_1)}}$$

Error Back Propagation Procedure

Error Back Propagation Procedure

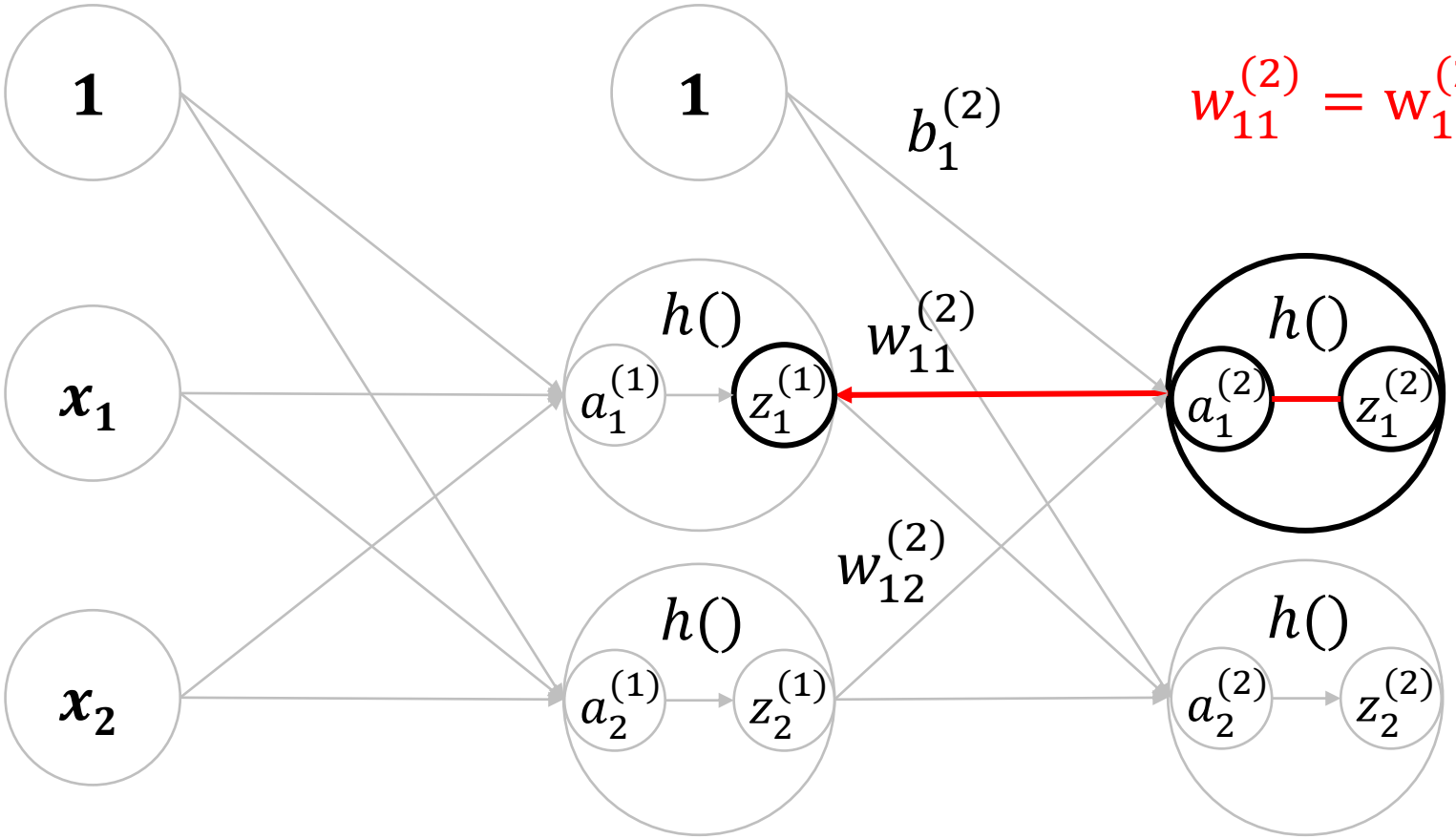


$$J_{Total} = J_1 + J_2$$

$$J_1 = \frac{1}{2}\left(z_1^{(2)} - y_1\right)^2$$

$$J_2 = \frac{1}{2}\left(z_2^{(2)} - y_2\right)^2$$

Error Back Propagation Procedure



$$w_{11}^{(2)} = \mathrm{w}_{11}^{(2)} - \eta \frac{dJ_{total}}{d\mathrm{w}_{11}^{(2)}}$$

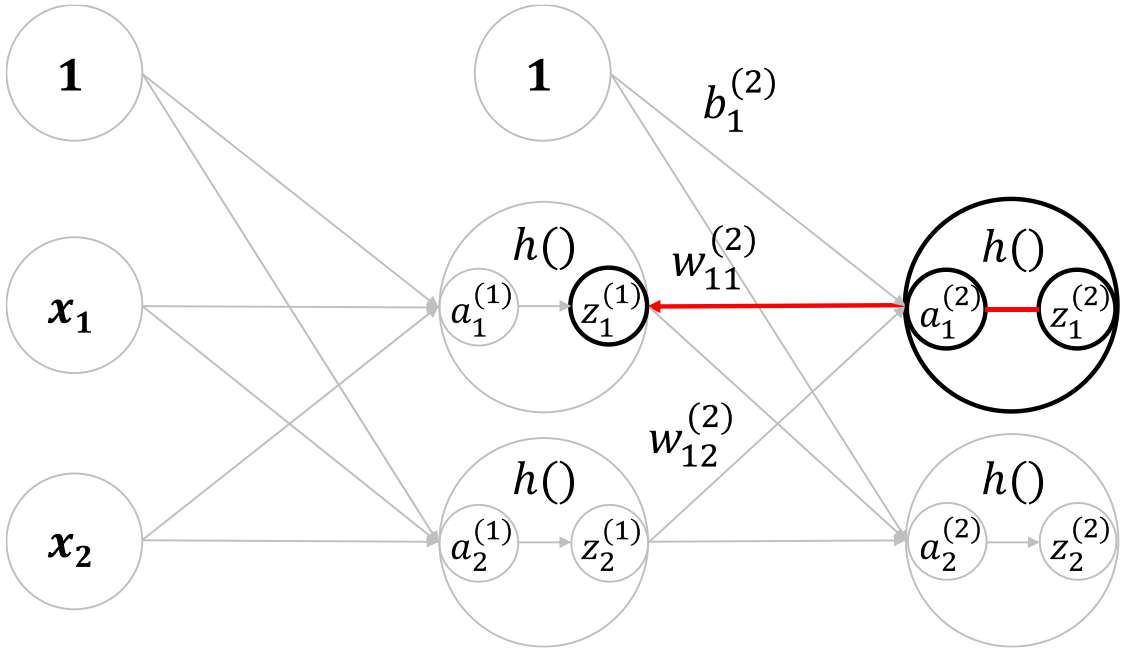
Error Back Propagation Procedure

$$\frac{dJ_{total}}{dw_{11}^{(2)}} = \frac{dJ_1}{dz_1^{(2)}} \frac{dz_1^{(2)}}{da_1^{(2)}} \frac{da_1^{(2)}}{dw_{11}^{(2)}}$$

$$J_1 = \frac{1}{2}\left(z_1^{(2)} - y_1\right)^2$$

$$z_1^{(2)} = h\left(a_1^{(2)}\right)$$

$$a_1^{(2)} = w_{11}^{(2)} z_1^{(1)} + w_{12}^{(2)} z_2^{(1)} + b_1^{(2)}$$



Error Back Propagation Procedure

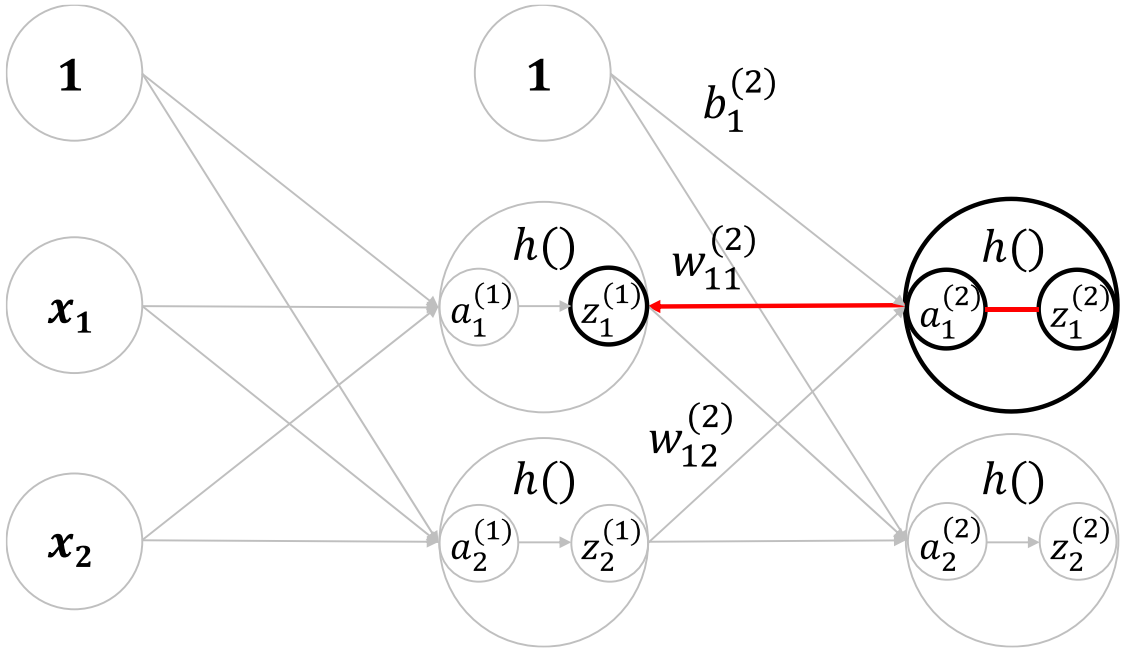
$$\frac{dJ_{total}}{dw_{11}^{(2)}} = \frac{dJ_1}{dz_1^{(2)}} \frac{dz_1^{(2)}}{da_1^{(2)}} \frac{da_1^{(2)}}{dw_{11}^{(2)}}$$

$$\frac{dJ_1}{dz_1^{(2)}} = \left(\frac{1}{2}\right)(2)\left(z_1^{(2)} - y_1\right) = \left(z_1^{(2)} - y_1\right)$$

$$J_1 = \frac{1}{2}\left(z_1^{(2)} - y_1\right)^2$$

$$z_1^{(2)} = h\left(a_1^{(2)}\right)$$

$$a_1^{(2)} = w_{11}^{(2)} z_1^{(1)} + w_{12}^{(2)} z_2^{(1)} + b_1^{(2)}$$



Error Back Propagation Procedure

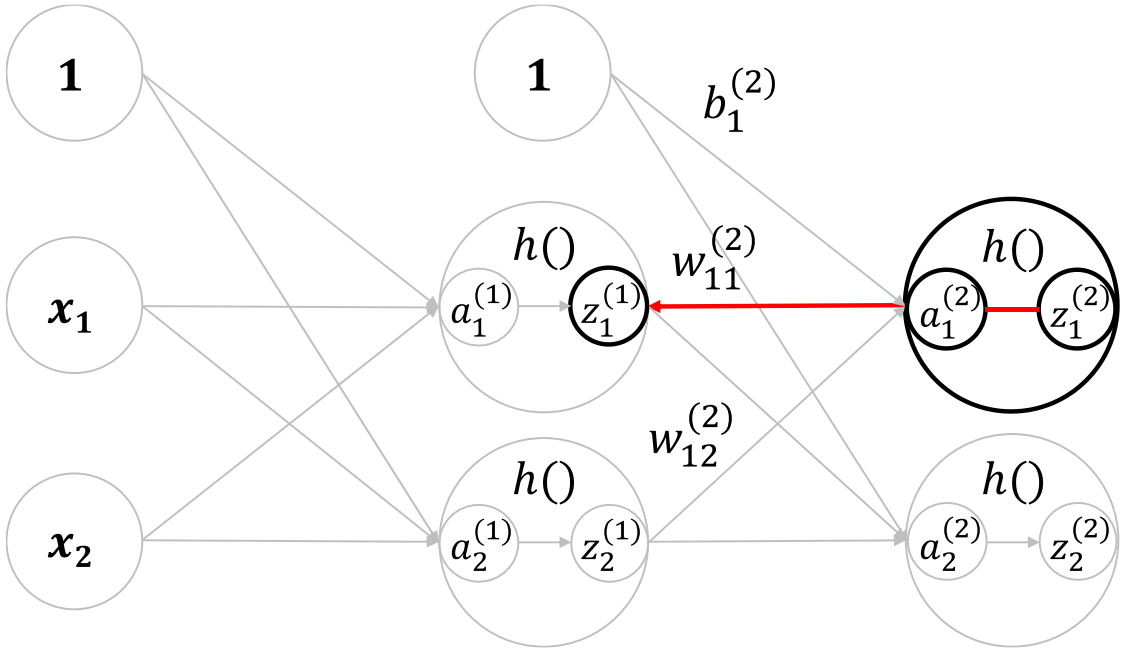
$$\frac{dJ_{total}}{dw_{11}^{(2)}} = \frac{dJ_1}{dz_1^{(2)}} \frac{dz_1^{(2)}}{da_1^{(2)}} \frac{da_1^{(2)}}{dw_{11}^{(2)}}$$

$$\frac{dz_1^{(2)}}{da_1^{(2)}} = h\left(a_1^{(2)}\right)\left(1 - h\left(a_1^{(2)}\right)\right) = z_1^{(2)}\left(1 - z_1^{(2)}\right)$$

$$J_1 = \frac{1}{2}\left(z_1^{(2)} - y_1\right)^2$$

$$z_1^{(2)} = h\left(a_1^{(2)}\right)$$

$$a_1^{(2)} = w_{11}^{(2)} z_1^{(1)} + w_{12}^{(2)} z_2^{(1)} + b_1^{(2)}$$



Error Back Propagation Procedure

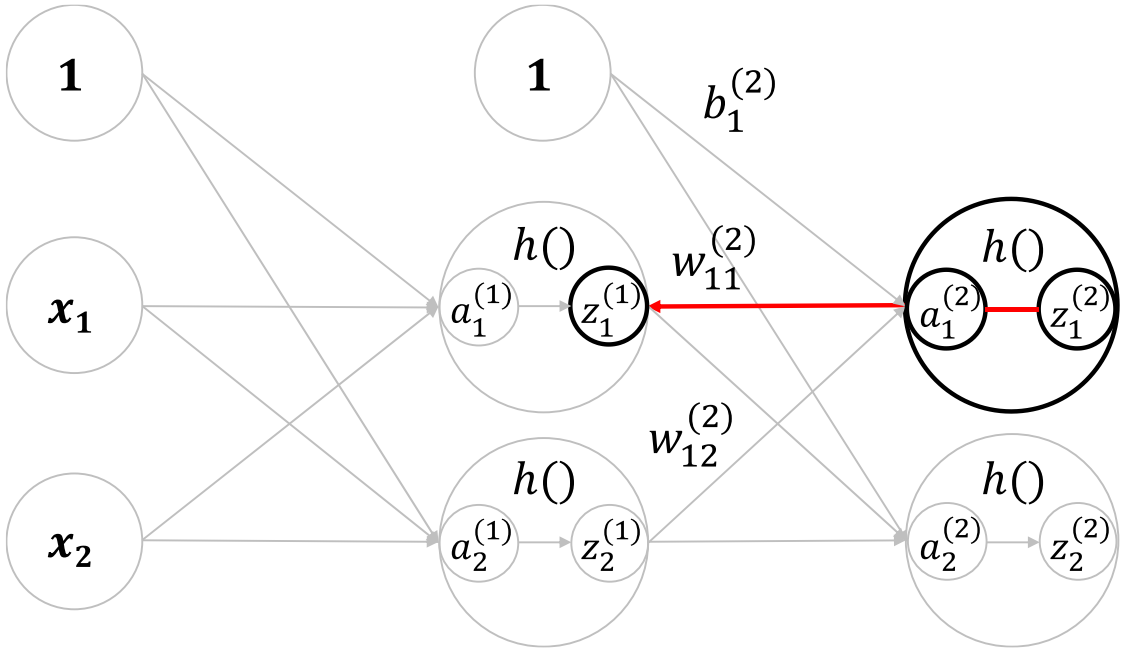
$$\frac{dJ_{total}}{dw_{11}^{(2)}} = \frac{dJ_1}{dz_1^{(2)}} \frac{dz_1^{(2)}}{da_1^{(2)}} \underbrace{\frac{da_1^{(2)}}{dw_{11}^{(2)}}}$$

$$\frac{da_1^{(2)}}{dw_{11}^{(2)}} = z_1^{(1)}$$

$$J_1 = \frac{1}{2}\left(z_1^{(2)} - y_1\right)^2$$

$$z_1^{(2)} = h\left(a_1^{(2)}\right)$$

$$a_1^{(2)} = w_{11}^{(2)} z_1^{(1)} + w_{12}^{(2)} z_2^{(1)} + b_1^{(2)}$$

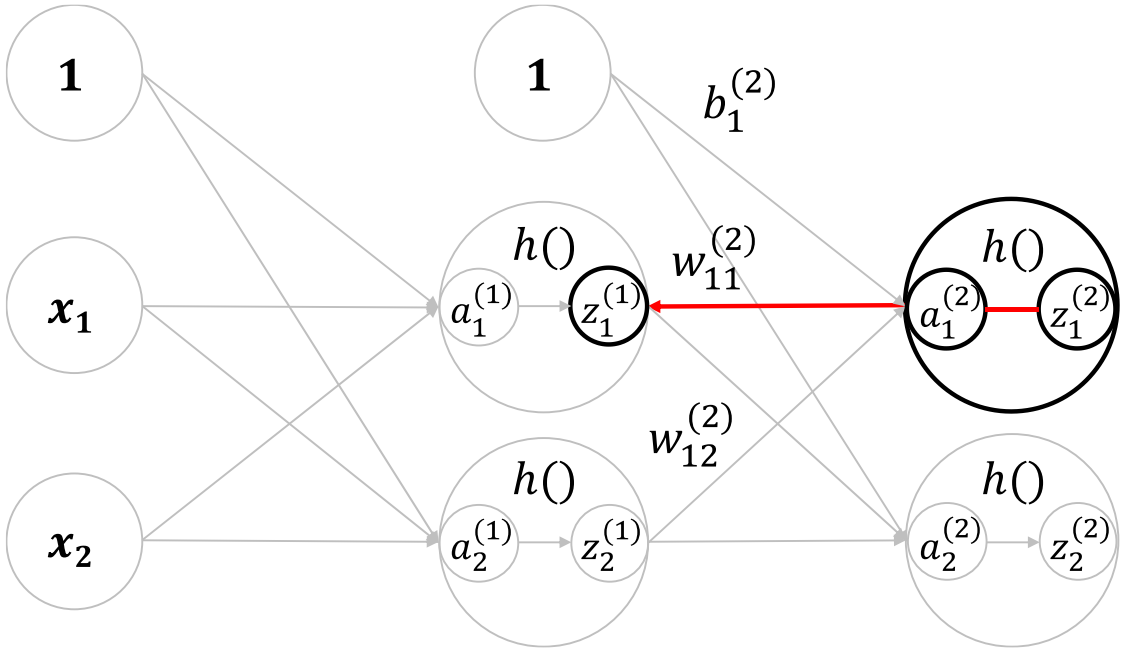


Error Back Propagation Procedure

$$\frac{dJ_{total}}{dw_{11}^{(2)}} = \frac{dJ_1}{dz_1^{(2)}} \frac{dz_1^{(2)}}{da_1^{(2)}} \frac{da_1^{(2)}}{dw_{11}^{(2)}}$$

$$= \left(z_1^{(2)} - y_1\right) z_1^{(2)} \left(1 - z_1^{(2)}\right) z_1^{(1)}$$

$$= \delta_1^{(2)} z_1^{(1)}$$

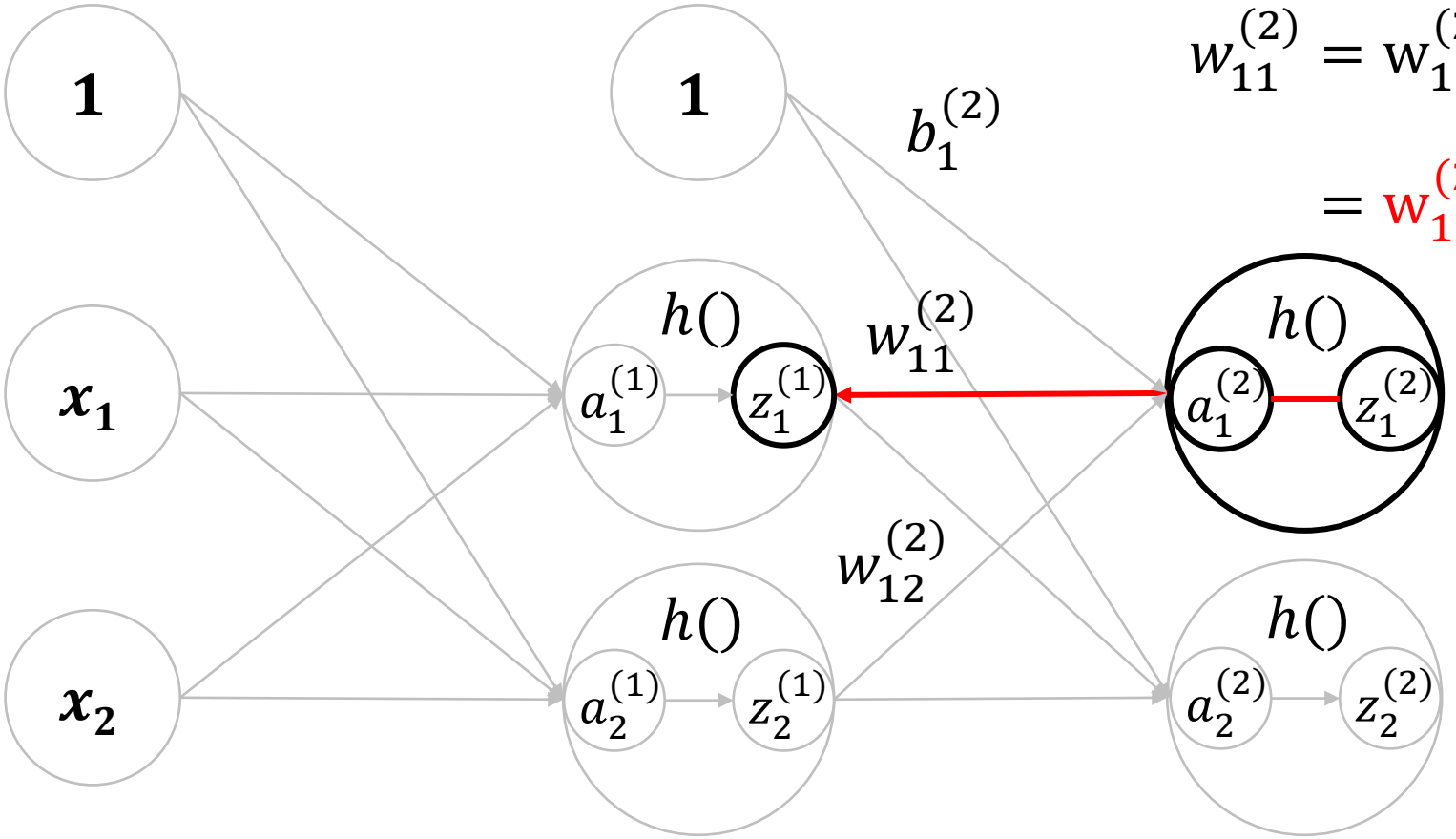


$$\frac{dJ_1}{dz_1^{(2)}} = \left(z_1^{(2)} - y_1\right)$$

$$\frac{dz_1^{(2)}}{da_1^{(2)}} = z_1^{(2)}\left(1 - z_1^{(2)}\right)$$

$$\frac{da_1^{(2)}}{dw_{11}^{(2)}} = z_1^{(1)}$$

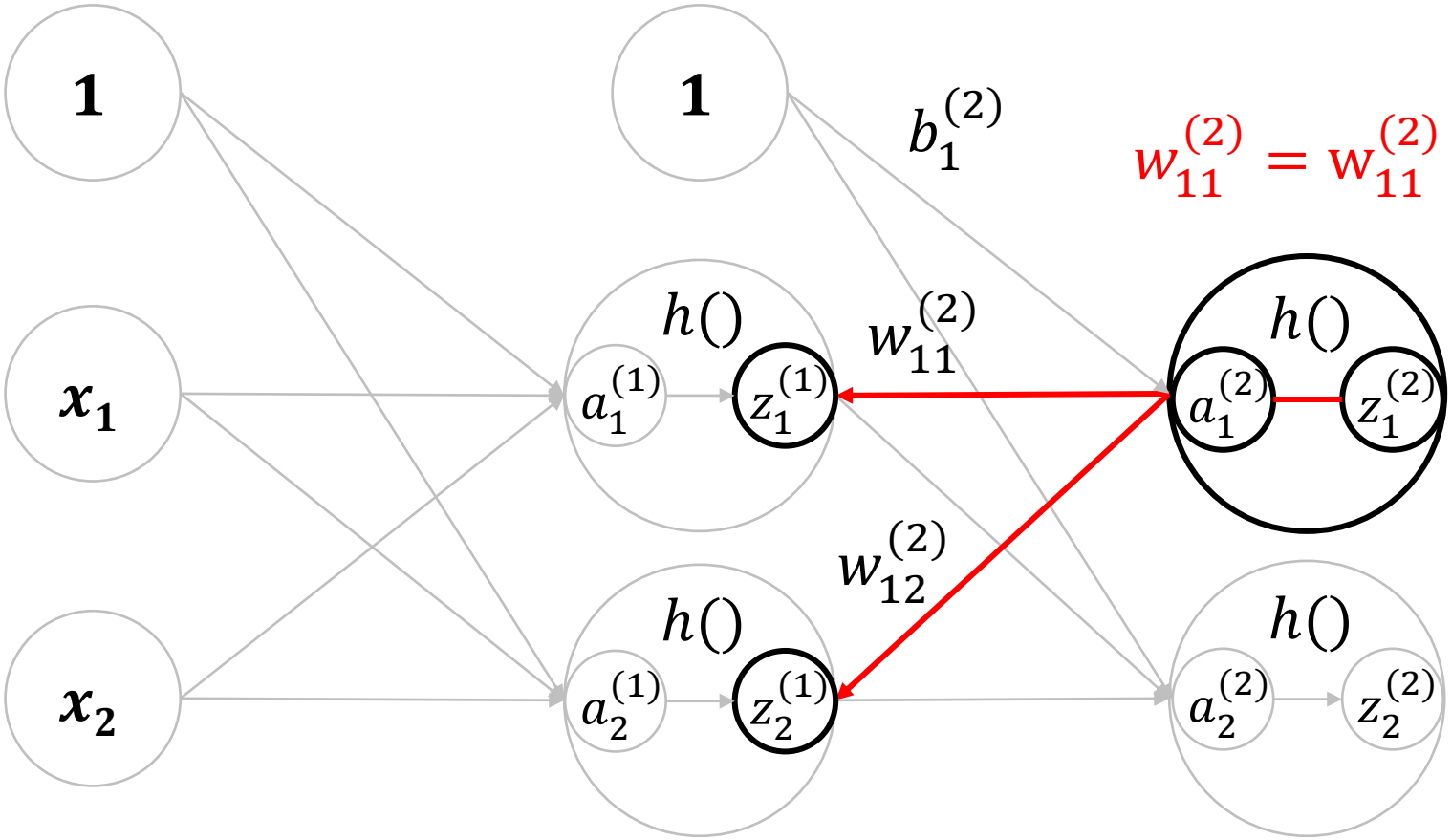
Error Back Propagation Procedure ($\eta = 1$)



$$w_{11}^{(2)} = w_{11}^{(2)} - \eta \frac{dJ_{total}}{dw_{11}^{(2)}}$$

$$= w_{11}^{(2)} - \delta_1^{(2)} z_1^{(1)}$$

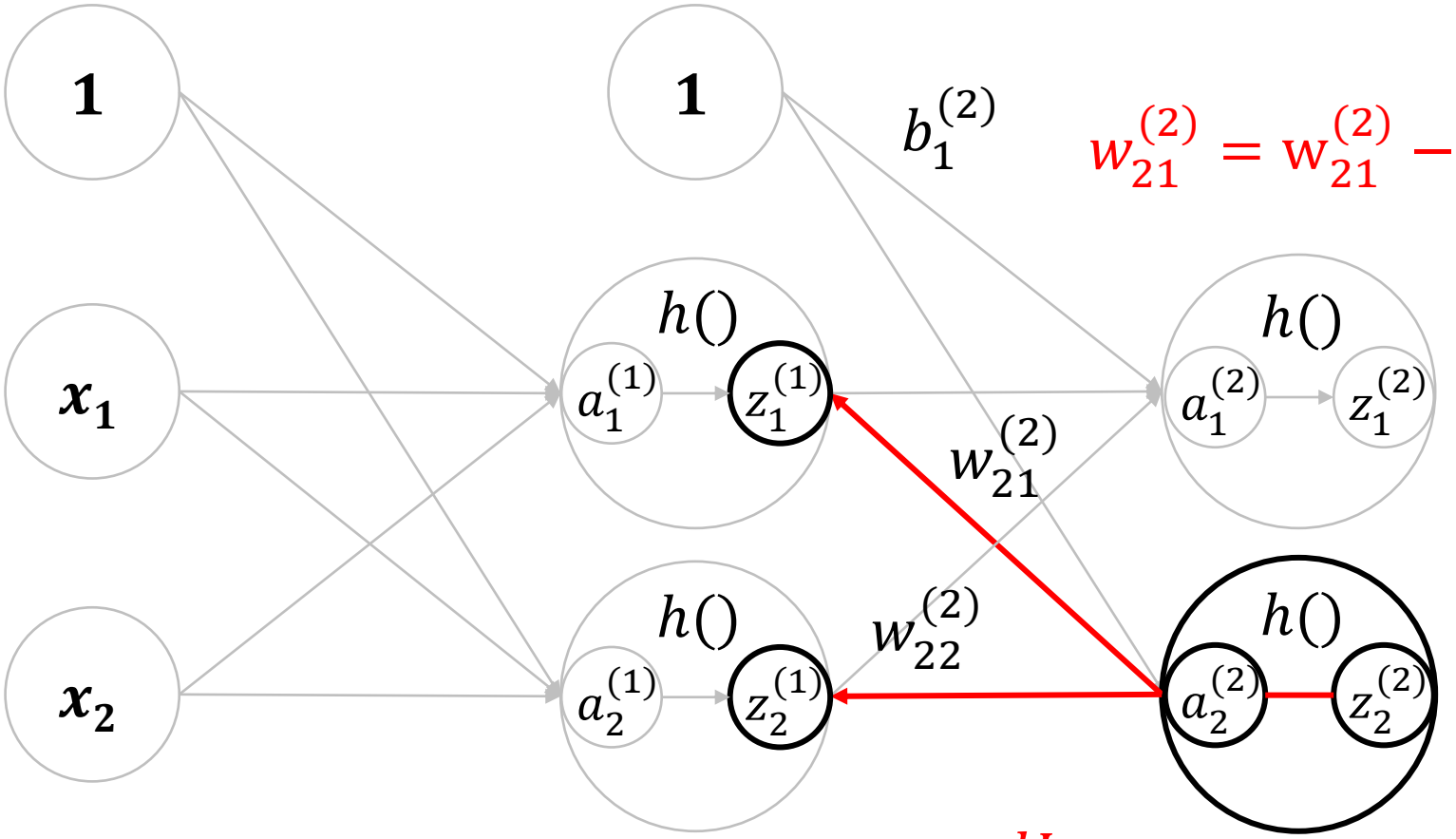
Error Back Propagation Procedure



$$w_{11}^{(2)} = w_{11}^{(2)} - \delta_1^{(2)} z_1^{(1)}$$

$$w_{12}^{(2)} = w_{12}^{(2)} - \delta_1^{(2)} z_2^{(1)}$$

Error Back Propagation Procedure



$$w_{21}^{(2)} = \mathrm{w}_{21}^{(2)} - \eta \frac{dJ_{total}}{d\mathrm{w}_{21}^{(2)}}$$

$$w_{22}^{(2)} = \mathrm{w}_{22}^{(2)} - \eta \frac{dJ_{total}}{d\mathrm{w}_{22}^{(2)}}$$

Error Back Propagation Procedure

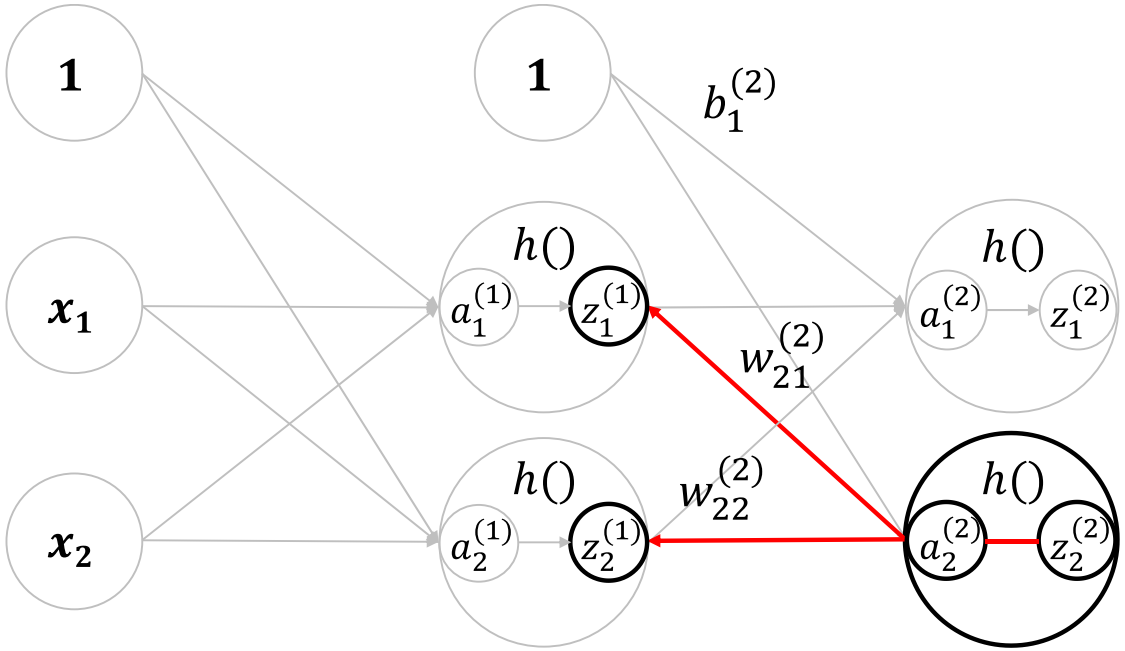
$$\frac{dJ_{total}}{dw_{21}^{(2)}} = \frac{dJ_2}{dz_2^{(2)}} \frac{dz_2^{(2)}}{da_2^{(2)}} \frac{da_2^{(2)}}{dw_{21}^{(2)}}$$

$$\frac{dJ_{total}}{dw_{22}^{(2)}} = \frac{dJ_2}{dz_2^{(2)}} \frac{dz_2^{(2)}}{da_2^{(2)}} \frac{da_2^{(2)}}{dw_{22}^{(2)}}$$

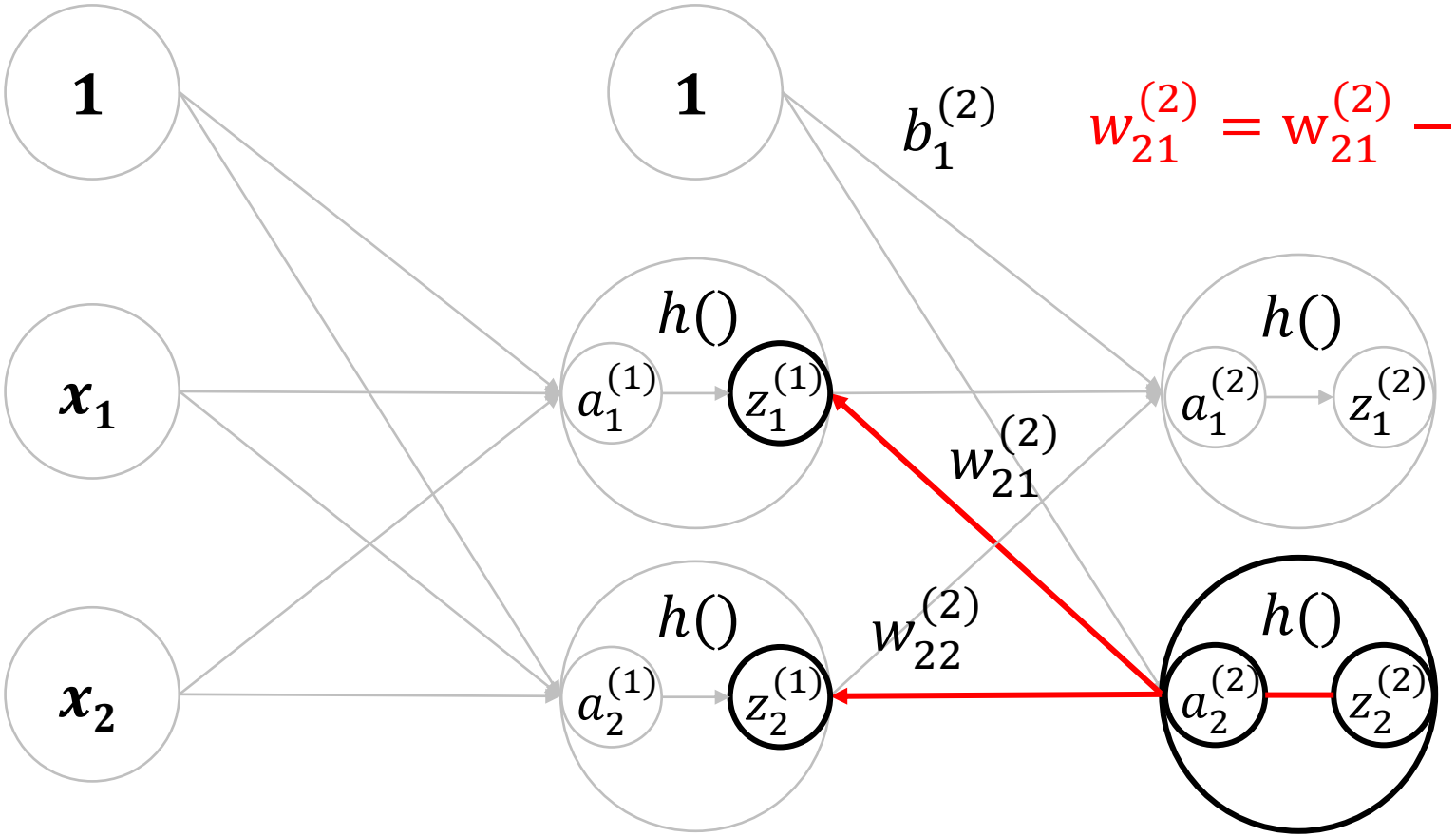
$$J_2 = \frac{1}{2}\left(z_2^{(2)} - y_2\right)^2$$

$$z_2^{(2)} = h\left(a_2^{(2)}\right)$$

$$a_2^{(2)} = w_{21}^{(2)} z_1^{(1)} + w_{22}^{(2)} z_2^{(1)} + b_1^{(2)}$$



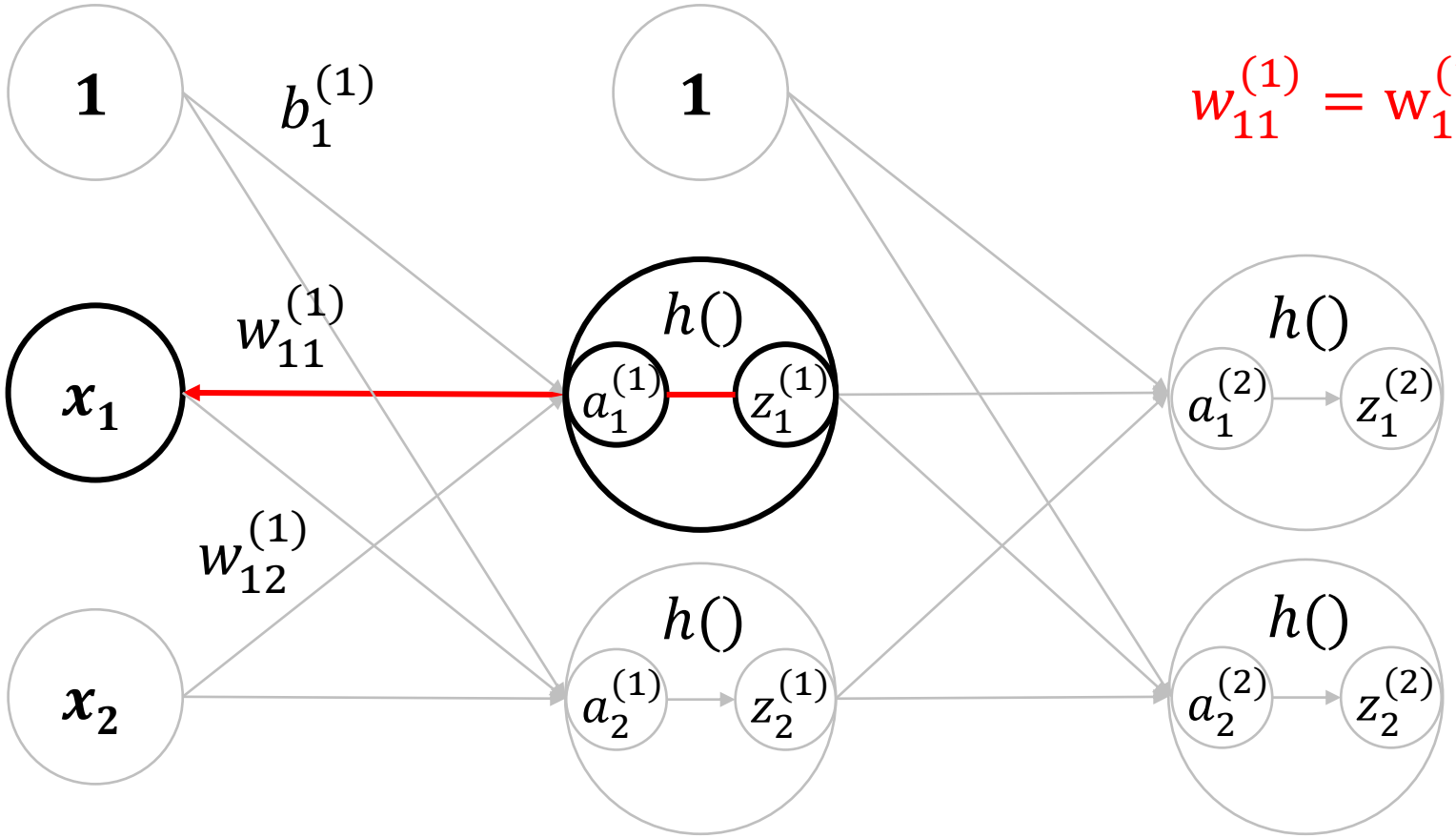
Error Back Propagation Procedure



$$w_{21}^{(2)} = w_{21}^{(2)} - \delta_2^{(2)} z_1^{(1)}$$

$$w_{22}^{(2)} = w_{22}^{(2)} - \delta_2^{(2)} z_2^{(1)}$$

Error Back Propagation Procedure

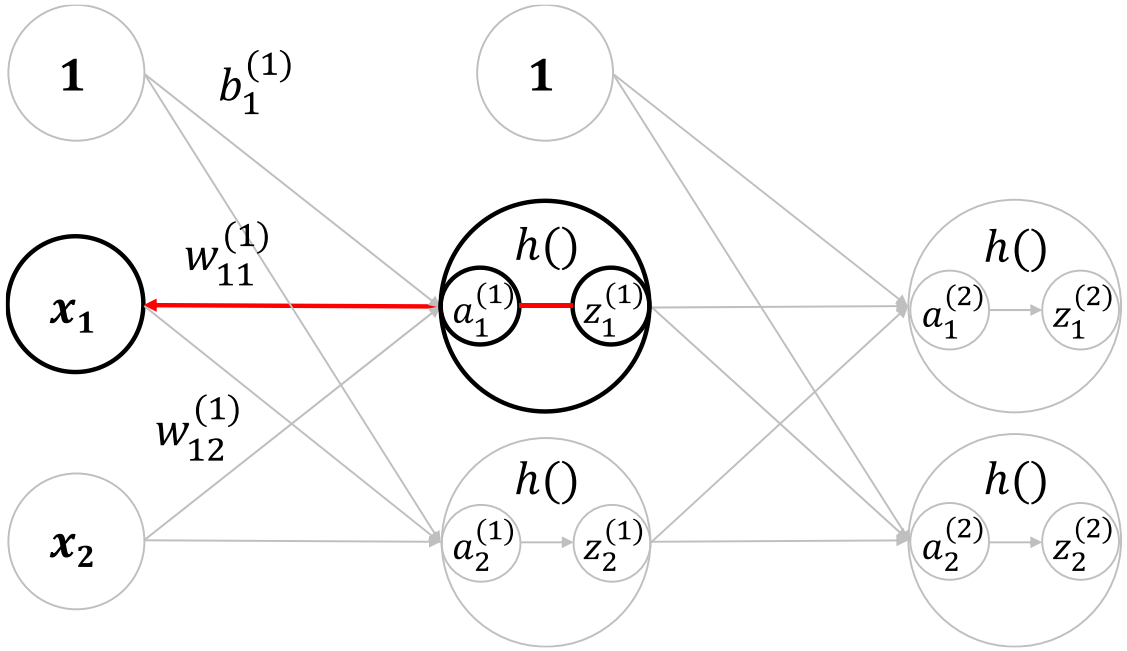


$$w_{11}^{(1)} = \mathrm{w}_{11}^{(1)} - \eta \frac{dJ_{total}}{d\mathrm{w}_{11}^{(1)}}$$

Error Back Propagation Procedure

$$\frac{dJ_{total}}{dw_{11}^{(1)}} = \frac{dJ_{total}}{dz_1^{(1)}} \frac{dz_1^{(1)}}{da_1^{(1)}} \frac{da_1^{(1)}}{dw_{11}^{(1)}}$$

$$J_1 = \frac{1}{2}\left(z_1^{(2)} - y_1\right)^2$$
$$J_2 = \frac{1}{2}\left(z_2^{(2)} - y_2\right)^2$$
$$z_1^{(1)} = h\left(a_1^{(1)}\right)$$
$$a_1^{(1)} = w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + b_1^{(1)}$$



Error Back Propagation Procedure

$$\frac{dJ_{total}}{dw_{11}^{(1)}} = \frac{dJ_{total}}{dz_1^{(1)}} \frac{dz_1^{(1)}}{da_1^{(1)}} \frac{da_1^{(1)}}{dw_{11}^{(1)}}$$

$$\frac{dJ_{total}}{dz_1^{(1)}} = \frac{dJ_1}{dz_1^{(1)}} + \frac{dJ_2}{dz_1^{(1)}} = \frac{dJ_1}{da_1^{(2)}} \frac{da_1^{(2)}}{dz_1^{(1)}} + \frac{dJ_2}{da_2^{(2)}} \frac{da_2^{(2)}}{dz_1^{(1)}}$$

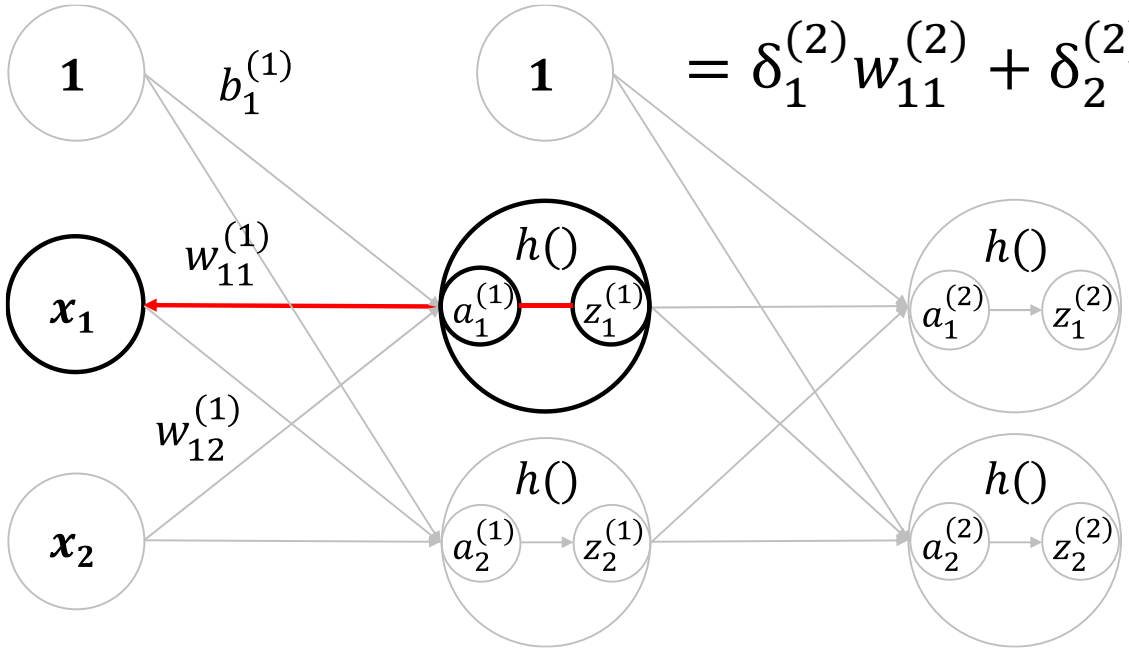
$$= \delta_1^{(2)} w_{11}^{(2)} + \delta_2^{(2)} w_{21}^{(2)}$$

$$\frac{dJ_1}{da_1^{(2)}} = \delta_1^{(2)}$$

$$\frac{dJ_2}{da_2^{(2)}} = \delta_2^{(2)}$$

$$a_1^{(2)} = w_{11}^{(2)} z_1^{(1)} + w_{12}^{(2)} z_2^{(1)} + b_1^{(2)}$$

$$a_2^{(2)} = w_{21}^{(2)} z_1^{(1)} + w_{22}^{(2)} z_2^{(1)} + b_1^{(2)}$$

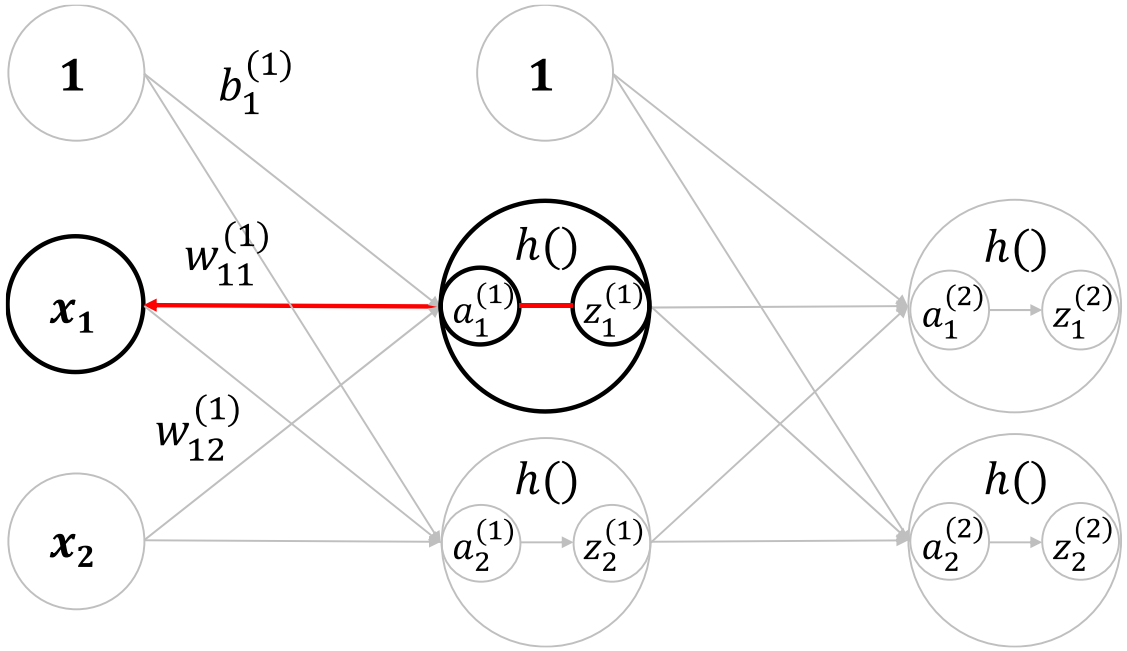


Error Back Propagation Procedure

$$\frac{dJ_{total}}{dw_{11}^{(1)}} = \frac{dJ_{total}}{dz_1^{(1)}} \frac{dz_1^{(1)}}{da_1^{(1)}} \frac{da_1^{(1)}}{dw_{11}^{(1)}}$$

$$\frac{dz_1^{(1)}}{da_1^{(1)}} = h\left(a_1^{(1)}\right)\left(1 - h\left(a_1^{(1)}\right)\right) = z_1^{(1)}\left(1 - z_1^{(1)}\right)$$

$$z_1^{(1)} = h\left(a_1^{(1)}\right)$$

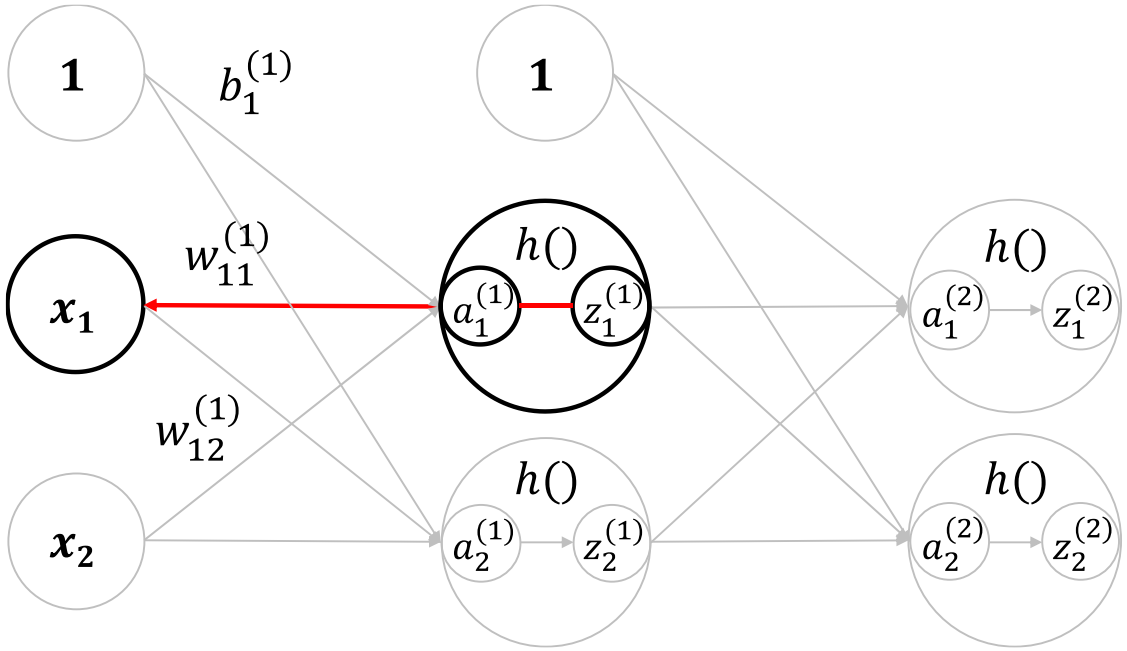


Error Back Propagation Procedure

$$\frac{dJ_{total}}{dw_{11}^{(1)}} = \frac{dJ_{total}}{dz_1^{(1)}} \frac{dz_1^{(1)}}{da_1^{(1)}} \underbrace{\frac{da_1^{(1)}}{dw_{11}^{(1)}}}$$

$$\frac{da_1^{(1)}}{dw_{11}^{(1)}} = x_1$$

$$\boxed{a_1^{(1)} = w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + b_1^{(1)}}$$

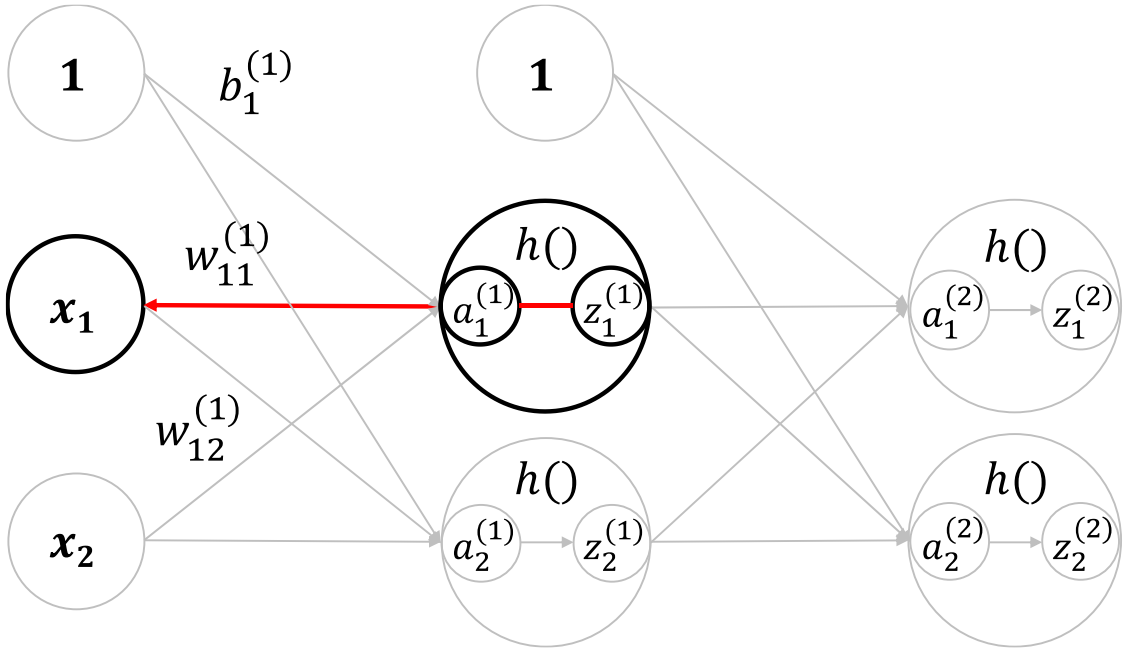


Error Back Propagation Procedure

$$\frac{dJ_{total}}{dw_{11}^{(1)}} = \frac{dJ_{total}}{dz_1^{(1)}}\frac{dz_1^{(1)}}{da_1^{(1)}}\frac{da_1^{(1)}}{dw_{11}^{(1)}}$$

$$= (\delta_1^{(2)} w_{11}^{(2)} + \delta_2^{(2)} w_{21}^{(2)})(z_1^{(1)}\left(1 - z_1^{(1)}\right))x_1$$

$$= \delta_1^{(1)}\, x_1$$



$$\frac{dJ_{total}}{dz_1^{(1)}} = \delta_1^{(2)} w_{11}^{(2)} + \delta_2^{(2)} w_{21}^{(2)}$$

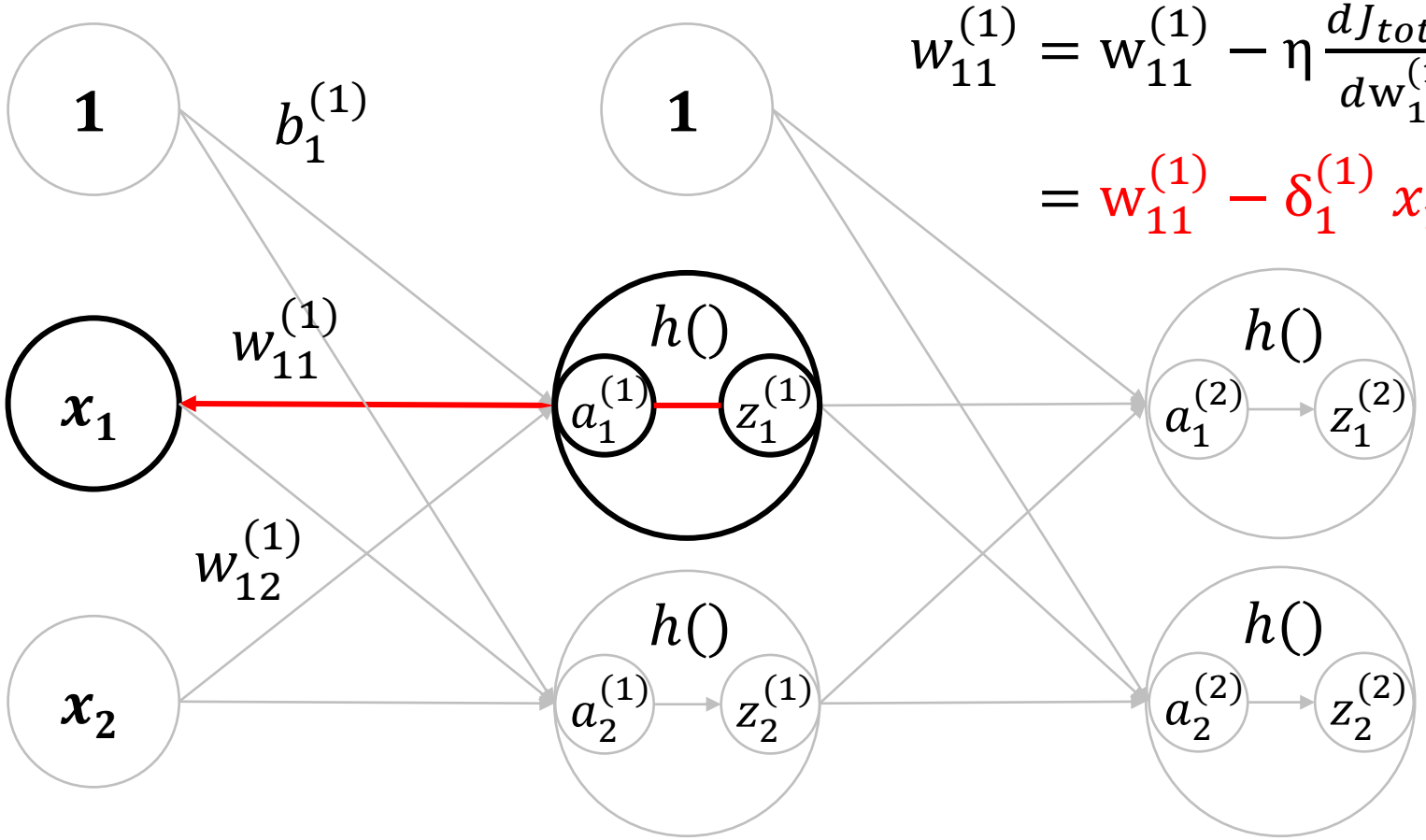
$$\frac{dz_1^{(1)}}{da_1^{(1)}} = z_1^{(1)}\left(1 - z_1^{(1)}\right)$$

$$\frac{da_1^{(1)}}{dw_{11}^{(1)}} = x_1$$

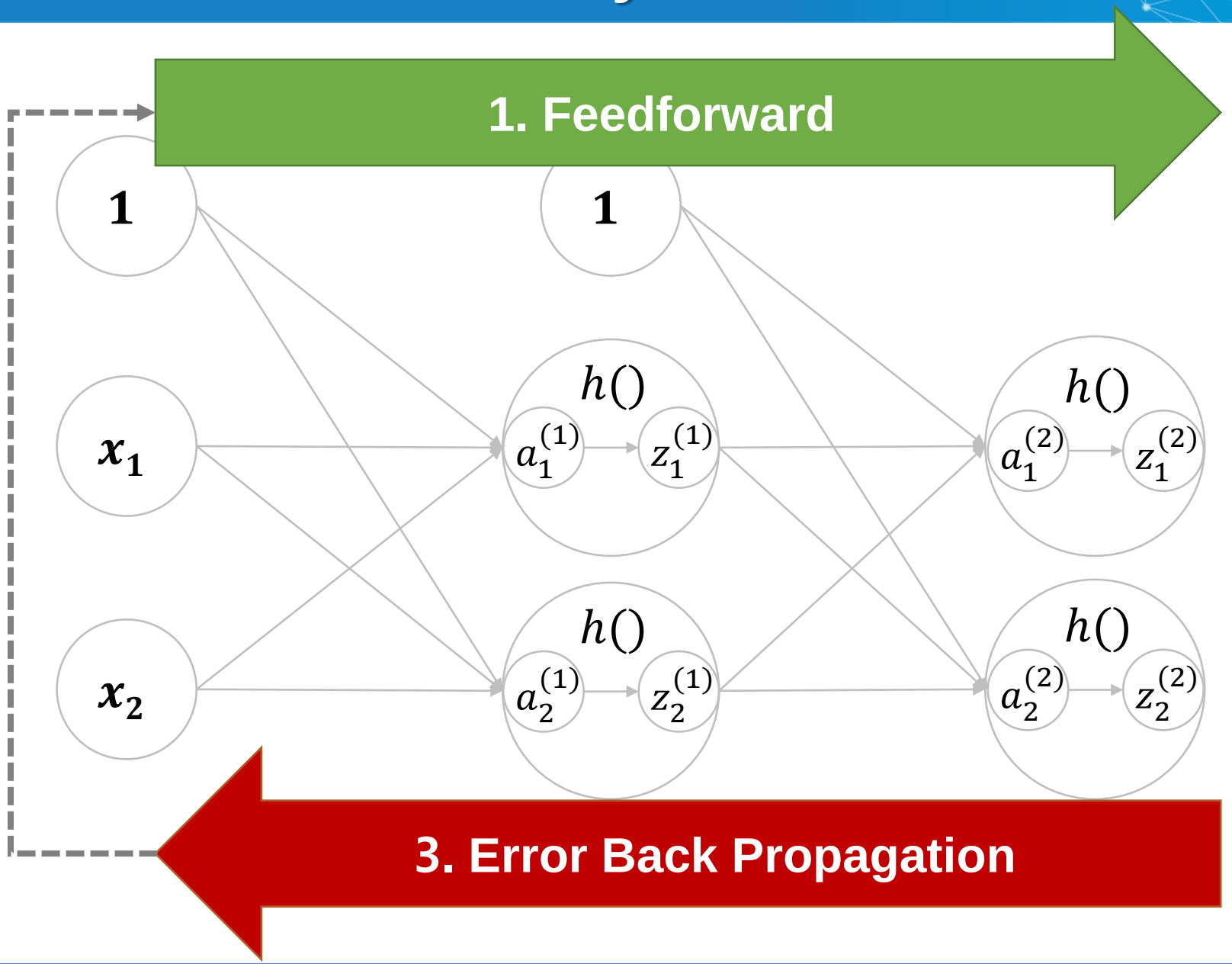
Error Back Propagation Procedure ($\eta = 1$)

$$w_{11}^{(1)} = \mathrm{w}_{11}^{(1)} - \eta \frac{dJ_{total}}{d\mathrm{w}_{11}^{(1)}}$$

$$= \mathrm{w}_{11}^{(1)} - \delta_1^{(1)} x_1$$



Procedure Summary



2. Loss calculation

교육부 Ministry of Education

NRF 한국연구재단 National Research Foundation of Korea

thank you

본 과제(결과물)는 교육부와 한국연구재단의 재원으로 지원을 받아 수행된
디지털신기술인재양성 혁신공유대학사업의 연구결과입니다.

Convergence and Open Sharing System COSS 인공지능 · 전남대학교 CHONNAM NATIONAL UNIVERSITY · 성균관대학교 SUNGKYUNKWAN UNIVERSITY · KNU 경북대학교 KYUNGPOOK NATIONAL UNIVERSITY · 전주대학교 JEONJU UNIVERSITY · 서울시립대학교 UNIVERSITY OF SEOUL · 서울과학기술대학교 SEOUL NATIONAL UNIVERSITY OF SCIENCE & TECHNOLOGY · 영진전문대학교 YEUNGJIN UNIVERSITY